

Lab Assignment Guidance

Generated 10/7/2026, 6:05:25 PM • 208 messages • Source: Shared ChatGPT link
<https://chatgpt.com/share/6ac63c53-f864-83e8-9d12-9d1f7d9aedac>

USER

https://github.com/SETAPESU26/61_math_flashcards hey this is my lab 4 assignment help me to do this and explain every thing how to do this step by step and i dont know any thing in that tell everything how to do how to download and who to open ,how to record and what i want to record every thing you want to explain to me please explain every detail things

ASSISTANT

Yes. I can guide you **from absolute zero**, including downloading the GitHub project, opening it, running it, recording the “before” video, using ChatGPT/Vibe Coding to fix it, adding every feature from the README, making separate commits, pushing to your own repository, and finally uploading everything into the `Lab-4` folder.

I checked your Lab 4 handout. The important requirement is: **work individually, clone/fork the assigned repo, run the Python game/puzzle, record 10 seconds before changes, use prompts to fix/add features, record 10 seconds after, push to your repo, and make separate commits for each task.**

We will do it ONE STEP at a time

I won't give you 20 steps at once and confuse you.

Our process will be:

1. Understand your assigned GitHub repository
2. Install/check Git
3. Install/check Python
4. Download/clone the repository
5. Open it in VS Code
6. Read the README and understand exactly what your lab requires
7. Run the original/broken program
8. Record the 10-second BEFORE video
9. Make the first fix using a good Vibe Coding prompt
10. Test it
11. Commit Task 1
12. Add Feature 1
13. Test!' commit
14. Add Feature 2
15. Test!' commit
16. Continue until every README requirement is completed
17. Record the 10-second AFTER video
18. Push everything to your own GitHub repository
19. Create the required `Lab-4` folder
20. Put:
 - Before video
 - After video

- Updated code
- Exported ChatGPT chat history

21. Final check before submission

The handout specifically says the final **Lab -4** folder must contain **videos, updated code, and exported chat history as a document/PDF**.

STEP 1 — Open your assigned GitHub repository

Your assigned repository is:

1. Open Chrome

Go to:

Then open your repository using the link above.

You should see something similar to:

TEXT

61_math_flashcards

README.md

...

2. DON'T change anything yet

Very important: At this stage, don't edit any files and don't try to fix the code.

We first need to understand what your teacher has given you.

3. Find **README.md**

Inside the GitHub repository, look for:

TEXT

README.md

Click it.

The README is extremely important because your handout says to **go through the README to understand the deliverables**, and then add the features listed there one by one.

4. Send me a screenshot

After opening the repository, send me **a screenshot of the main GitHub repository page**.

If you can see the README contents, send that screenshot too.

Don't download or install anything yet.

Once you send the screenshot, I'll give you **STEP 2 only**, and we'll continue together until the entire Lab 4 is finished.

USER

Math Flashcards Repair Lab

This project is an interactive mental arithmetic flashcard game using Pygame. It introduces students to mathematical operation parsing, string formatting vs arithmetic evaluation, custom text-box input components, and feedback messaging within an object-oriented codebase.

What's Provided

A working Math Flashcards game with:

- Dynamic flashcard generation featuring randomized operands and operators (+, -, *)
- Subtraction safety logic preventing negative outcomes during card generation
- A custom numeric `TextBox` widget supporting cursor input, digit entry, and backspace
- Input submission via the `Return` / `Enter` key or clicking the `SUBMIT` button
- Live score and attempt tracking with color-coded feedback messages

It has **one deliberate bug** and **three optional features** left as tasks to implement. You are expected to **analyze**, **interact with an AI assistant**, and **complete/fix** the game to make it fully functional and more interesting.

Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab.

Getting Started

Setup

1. Make sure you have Python 3.10+ installed.
2. Install dependencies:

```
pip install pygame
```

3. Run the game:

```
python main.py
```

Controls: Type numeric digits into the text box and press Return (or click SUBMIT) to submit your answer

Tasks to Complete

Each task must be completed using an iterative process involving LLM suggestions and your critical code review.

Task 1: Fix the string concatenation calculation bug

Submitting the mathematically correct answer is rejected as incorrect because operands are joined together as text instead of being calculated. Ensure the game correctly evaluates the arithmetic problem according to its active operator.

Task 2: Implement a per-question timer bar

Players currently have unlimited time to answer each flashcard. Add a visible countdown timer bar below the active card that tracks remaining time and automatically registers a missed attempt if time runs out before submission.

Task 3: Implement consecutive correct streak multipliers

Correct answers currently grant only a flat point increase regardless of player consistency. Introduce a streak multiplier that escalates score rewards for consecutive correct answers and resets back to baseline on any wrong answer or timeout.

Task 4: Implement Division Operator with Clean Integer Quotients

The flashcard pool only tests addition, subtraction, and multiplication. Add integer division into card generation, ensuring that every generated problem divides evenly with whole integer answers and no remainders.

Expected Behavior

- Flashcards present arithmetic problems with random numbers and operators.
- Typing the correct arithmetic answer increments the score and moves to the next card.
- Submitting an incorrect answer displays the expected value and clears the input box for retry or tracking.
- Submitting an empty input box prompts the user without counting as a failed attempt.

Folder Structure

```
math_flashcards/  
% % %   g a m e /  
%       % % %   g a m e _ e n g i n e . p y  
%       % % %   t e x t _ b o x . p y  
% % %   m a i n . p y  
% % %   R E A D M E . m d
```

Submission Checklist

Submission is only the following three things:

- ☐ A 10-second video of gameplay **before** your changes, showing the bug/broken behavior
- ☐ A 10-second video of gameplay **after** your changes, showing the bug fixed and the new features working
- ☐ The Chat/LLM used page link, with the complete chat history

ASSISTANT

Perfect. ~~Now~~ we know **exactly what your Lab 4 requires**. You have **4 tasks** to complete, not just one bug.

We will do this slowly, **one step at a time**, exactly as you requested. Don't start coding yet.

What your Lab 4 is asking you to do

Your original game already has:

- Addition ☐
- Subtraction ☐
- Multiplication ☐
- Random questions
- Answer textbox
- Submit button
- Score
- Attempts
- Feedback

You have to complete:

Task	What you have to do
Task 1	Fix the arithmetic calculation bug
Task 2	Add a countdown timer bar
Task 3	Add consecutive-correct streak multiplier
Task 4	Add division $\div$ with only whole-number answers

And importantly, your teacher wants you to use an **LLM such as ChatGPT/Claude as your debugging/pair-programming partner**, rather than simply copying a finished solution.

Before we start — VERY IMPORTANT

Your lab specifically says:

Record a 10-second video before making any changes.

So **we must not modify the code before recording the original behavior.**

We need to first:

Download !' Open !' Run !' Demonstrate bug !' Record

Only after that will we start Task 1.

STEP 2 — Check Python on your computer

Before downloading the project, let's check whether Python is already installed.

1. Open Windows Terminal

On your keyboard press:

Windows key + R

A small box will appear.

Type:

```
TEXT
cmd
```

and press **Enter**.

You will get something like:

```
TEXT
C:\Users\YourName>
```

2. Type this command

BASH

```
python --version
```

Press **Enter**.

You may get one of these:

Case A — Python is installed

You might see:

TEXT

```
Python 3.12.4
```

or:

TEXT

```
Python 3.11.9
```

That's good.

Your README requires **Python 3.10+**, so Python 3.10, 3.11, 3.12, etc. are acceptable.

Case B — Python isn't installed

You might see something like:

TEXT

```
'python' is not recognized as an internal or external command
```

That's okay. We'll install it.

Don't do anything else yet

Do only this:

1. Press **Windows + R**
2. Type **cmd**
3. Press **Enter**
4. Type:

BASH

```
python --version
```

5. Press **Enter**.
6. **Send me a screenshot of the result.**

I'll then tell you exactly what to do next based on what you get.

We will eventually also need to check **Git** and **VS Code**, but we'll do those one at a time.

USER

```
C:\Users\Admin>python --version Python 3.13.15
```

ASSISTANT

Perfect! ' **Python 3.13.15 is installed**, so you meet the lab's Python 3.10+ requirement.

We can move to the next check.

STEP 3 — Check Git

Git is what we will use later to:

- download/clone the GitHub project
- track each task separately
- create separate commits
- push your completed work to your own GitHub repository

1. In the same Command Prompt, type:

BASH

```
git --version
```

Then press **Enter**.

2. You should see something like:

TEXT

```
git version 2.51.0.windows.1
```

If it says Git is not recognized

For example:

TEXT

```
'git' is not recognized as an internal or external command
```

Don't worry. It simply means Git isn't installed, and I'll show you exactly how to install it.

For now, run only:

BASH

```
git --version
```

Send me what you get. Then we'll do the next step.

USER

```
mam told to do clone of that i dont know how to doo tell how to doo
```

ASSISTANT

Yes. Your mam specifically wants you to **clone the assigned GitHub repository**. I'll show you exactly how to do it. You don't need to know Git beforehand.

We will **not change any code yet**. First we'll clone the original repository to your computer.

STEP 3 — Clone the GitHub repository

Your assigned repository is:

Part A — First check whether Git is installed

You already have Command Prompt open.

Type:

```
BASH
git --version
```

and press **Enter**.

If you get something like:

```
TEXT
git version 2.51.0.windows.1
```

' Git is installed.

If you get:

```
TEXT
'git' is not recognized...
```

'LGit isn't installed. Tell me that result and I'll guide you through installing Git.

Part B — Create a place for your lab projects

Once Git is confirmed, we'll create a folder where the project will be downloaded.

In Command Prompt type:

```
BASH
cd Desktop
```

Press **Enter**.

Then:

```
BASH
mkdir Lab4
```

Press **Enter**.

Then:

```
BASH
cd Lab4
```

Press **Enter**.

Your Command Prompt should now look something like:

```
TEXT
C:\Users\Admin\Desktop\Lab4>
```

This is where we'll clone the project.

Part C — Clone the repository

Now type this **exactly**:

```
BASH
git clone https://github.com/SETAPESU26/61_math_flashcards.git
```

Then press **Enter**.

Git will download the repository.

You should see something similar to:

```
TEXT
Cloning into '61_math_flashcards'...
remote: Enumerating objects...
remote: Counting objects...
Receiving objects...
Resolving deltas...
```

Finally, you should get something like:

```
TEXT
done.
```

That means the repository has been cloned to your computer.

Part D — Check that it was downloaded

Type:

```
BASH
dir
```

You should see a folder:

TEXT

```
61_math_flashcards
```

So the structure on your computer will be approximately:

TEXT

Desktop

```
%% %   L a b 4
      %% %   6 1 _ m a t h _ f l a s h c a r d s
            %% %   g a m e
            %% %   m a i n . p y
            %% %   R E A D M E . m d
```

The README you gave me confirms that the project is supposed to have `game`, `main.py`, and `README.md`.

& Very important for your assignment

Do NOT start editing `main.py` yet.

The lab requires you to:

1. Clone the repository
2. Run the original game
3. Record the **10-second BEFORE video**
4. Then start fixing Task 1
5. Then Tasks 2, 3 and 4
6. Make separate commits
7. Push your completed work to your repository

The handout explicitly says to record the 10 seconds **before making changes**.

Your job right now

Do these **one at a time**:

\$`Check Git

BASH

```
git --version
```

\$`f Git works:

BASH

```
cd Desktop
```

BASH

```
mkdir Lab4
```

```
BASH
cd Lab4
```

\$!Clone:

```
BASH
git clone https://github.com/SETAPESU26/61_math_flashcards.git
```

\$!Check:

```
BASH
dir
```

Then **send me a screenshot of the Command Prompt after the clone finishes.**

I'll take you to the next step: **opening the cloned project in VS Code and running the original game.**

```
USER
C:\Users\Admin>git --version 'git' is not recognized as an internal or external command, operable program or batch file.
```

ASSISTANT

That's completely fine. **Git is not installed on your computer yet.** We need to install it before we can clone your lab repository.

Don't worry—we'll do it step by step.

STEP 4 — Install Git on Windows

1. Open the official Git website

Go here:

It should automatically start downloading the Windows installer.

You will get a file similar to:

```
TEXT
Git-2.xx.x-64-bit.exe
```

2. Open the downloaded installer

Go to your **Downloads** folder.

Double-click the Git installer:

```
TEXT
Git-...-64-bit.exe
```

Windows may ask:

Do you want to allow this app to make changes to your device?

Click:

Yes

STEP 5 — Install Git

The Git installation window will appear.

For this lab, you can safely keep the **default options**.

You will see several screens such as:

TEXT

Information
Select Destination Location
Select Components
Choosing the default editor
Adjusting your PATH environment
Choosing HTTPS transport backend
Configuring line ending conversions
Configuring terminal emulator

IMPORTANT

You don't need to change these settings.

For each screen, just use the default selection and click:

Next

until you reach:

Install

Click **Install**.

Wait for the installation to finish.

Then click:

Finish

STEP 6 — VERY IMPORTANT: Reopen Command Prompt

Close your current Command Prompt window.

Open a **new** Command Prompt:

Windows + R

Type:

TEXT

cmd

Press **Enter**.

Now type:

```
BASH
git --version
```

You should now see something like:

```
TEXT
git version 2.51.0.windows.1
```

The exact version number may be different. That's okay.

' If you see a Git version

Git is successfully installed.

Stop there and send me a screenshot.

Don't clone the repository yet. I'll guide you through the cloning step after we confirm Git is working.

ASSISTANT

Yes ~~0=You~~ are on the **correct official Git website**. I can see the Git for Windows download page in your screenshot.

Do this now

On the page you are showing, click:

"Click here to download the latest (2.56.0(2)) x64 version of Git for Windows."

That is the correct option for a normal Windows 64-bit computer.

It should download an installer, probably with a name similar to:

```
TEXT
Git-2.56.0.2-64-bit.exe
```

After it downloads

1. Open your **Downloads** folder.
2. Find the Git `.exe` installer.
3. Double-click it.
4. If Windows asks **"Do you want to allow this app to make changes?"**, click **Yes**.
5. The Git installation window will open.

Don't change any settings in the installer yet.

When the installer window opens, **send me a screenshot**. I'll tell you exactly what to click on each screen.

ASSISTANT

Perfect ~~0=You~~ are at the **Select Components** screen of Git installation.

What to do on this screen

Don't change anything. '

The options currently selected are fine for our lab.

Just click:

Next !'

You may get another screen after this.

Important

From this point onward, **send me a screenshot of each new installation screen before clicking anything**, and I'll tell you exactly what to select.

For this screen:

Click `Next` now.

Then send me the next screen.

ASSISTANT

Perfect. You are on the **Select Start Menu Folder** screen.

Everything is already correct.

Do this:

Leave:

TEXT

Git

exactly as it is.

Do **not** tick:

& Don't create a Start Menu folder

Just click:

Next !'

Then send me a screenshot of the **next screen**.

We'll continue one screen at a time.

ASSISTANT

Perfect. This screen is asking which **text editor Git should use**.

You currently have:

Use Vim (the ubiquitous text editor) as Git's default editor

Since you're a beginner, I recommend changing this to **Visual Studio Code** if it appears in the dropdown.

Do this:

1. Click the **dropdown arrow** on the right side of the box.
2. Look for:

Visual Studio Code

3. If you see it, select:

Visual Studio Code as Git's default editor

4. Then click **Next**.

If Visual Studio Code is NOT in the list

That's okay. **Don't change anything.**

Leave:

TEXT

Use Vim (the ubiquitous text editor) as Git's default editor

and click **Next**.

Send me a screenshot of the next screen. We'll continue from there.

ASSISTANT

Yes! You can see the dropdown clearly now.

Select:

``Use Visual Studio Code as Git's default editor``

It is the **4th option** in your dropdown.

Do this now:

1. Click **Use Visual Studio Code as Git's default editor**
2. Make sure the box now shows that option.
3. Click **Next**.

That's the best choice for you because we'll be working with the project in VS Code.

Then **send me the next screen** and I'll tell you exactly what to select there.

ASSISTANT

Perfect! This screen is:

"Adjusting the name of the initial branch in new repositories"

You have two choices.

Select this:

`' Let Git decide`

It is already selected in your screenshot.

Do not change anything.

Then click:

`'ip Next`

You may get another setup screen.

Send me the next screenshot, and I'll tell you exactly what to choose.

USER

```
C:\Users\Admin>git --version git version 2.56.0.windows.2
```

ASSISTANT

Excellent!  Git is installed correctly.

You have:

TEXT

```
git version 2.56.0.windows.2
```

So we can now **clone your lab repository**.

STEP 5 — Clone your Lab 4 repository

We will put the project on your **Desktop**.

1. Go to Desktop

In the same Command Prompt, type:

BASH

```
cd Desktop
```

Press **Enter**.

You should see:

TEXT

```
C:\Users\Admin\Desktop>
```

2. Create a folder called **Lab4**

Type:

BASH

```
mkdir Lab4
```

Press **Enter**.

3. Enter that folder

Type:

BASH

```
cd Lab4
```

Press **Enter**.

Now you should see:

TEXT

```
C:\Users\Admin\Desktop\Lab4>
```

4. Clone your teacher's repository

Now copy and paste this **exact command**:

BASH

```
git clone https://github.com/SETAPESU26/61_math_flashcards.git
```

Press **Enter**.

You should see messages such as:

TEXT

```
Cloning into '61_math_flashcards'...
remote: Enumerating objects...
remote: Counting objects...
Receiving objects...
Resolving deltas...
```

At the end, there should be no error.

5. Check that it downloaded

Type:

BASH

```
dir
```

You should see a folder named:

TEXT

```
61_math_flashcards
```

& Stop here

Don't open or edit the code yet.

Send me a screenshot showing the result of:

BASH

```
dir
```

Then I'll show you how to **open the cloned project in VS Code**, install Pygame, run the original game, and prepare the required **10-second BEFORE recording**.

USER

```
C:\Users\Admin\Desktop\Lab4>dir Volume in drive C has no label. Volume Serial Number is 6470-1D13
```

```
Directory of C:\Users\Admin\Desktop\Lab4
```

```
10/07/2026 03:20 PM <DIR>      . 10/07/2026 03:20 PM <DIR>      .. 10/07/2026 03:21 PM <DIR>
61_math_flashcards 0 File(s)      0 bytes 3 Dir(s) 156,145,250,304 bytes free
```

ASSISTANT

Excellent! ' **The clone was successful.**

Your output shows:

TEXT

```
C:\Users\Admin\Desktop\Lab4  
  
61_math_flashcards
```

So the repository is now downloaded to:

TEXT

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards
```

That means **the cloning part of your assignment is completed.** Ø<ß%

STEP 6 — Open the cloned project in VS Code

Now we need to open the **cloned copy**, not the GitHub website.

1. In Command Prompt

You are currently here:

TEXT

```
C:\Users\Admin\Desktop\Lab4>
```

Type:

BASH

```
cd 61_math_flashcards
```

Press **Enter**.

You should now see:

TEXT

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

2. Open this folder in VS Code

Now type:

BASH

```
code .
```

Press **Enter**.

The `.` means:

"Open the current folder."

VS Code should open with your `61_math_flashcards` project.

If `code .` works

You should see VS Code with files such as:

TEXT

```
61_math_flashcards
%
% % %   g a m e
%       % % %   g a m e _ e n g i n e . p y
%       % % %   t e x t _ b o x . p y
%
% % %   m a i n . p y
% % %   R E A D M E . m d
```

This matches the folder structure specified in your assignment.

If `code .` gives an error

For example:

TEXT

```
'code' is not recognized...
```

Don't worry. Send me the error and I'll show you another way to open the project.

& Important

Once VS Code opens, **DO NOT MODIFY ANY CODE YET.**

We still need to:

Open !' install Pygame !' run original game !' demonstrate Task 1 bug !' record BEFORE video.

Your lab specifically requires the before video **before making any changes.**

Do only this now:

BASH

```
cd 61_math_flashcards
```

then:

BASH

```
code .
```

If VS Code opens, **send me a screenshot of VS Code** showing the project files.

USER

in my laptop there is no vscode

ASSISTANT

No problem at all ~~Ø=VS~~ **VS Code is not installed on your laptop.** We need to install it before opening the cloned project.

Don't worry—we'll do this exactly like Git, one step at a time.

STEP 6 — Install Visual Studio Code

1. Open the official VS Code website

Go here:

2. Download VS Code

On the page, you should see a button like:

Download for Windows

Click it.

It will download an installer, something similar to:

TEXT

VSCodeUserSetup-x64-exe

3. Open the downloaded installer

Go to your **Downloads** folder.

Double-click the VS Code installer.

If Windows asks:

Do you want to allow this app to make changes to your device?

Click:

Yes

STEP 7 — VS Code installation

The installer will show a license agreement.

Choose:

' I accept the agreement

Then click:

Next

You can keep the default installation location.

Continue clicking **Next** until you reach:

"Select Additional Tasks"

When you reach this screen, **STOP**.

Send me a screenshot of that screen.

I'll tell you which boxes to tick.

Why we need VS Code

We'll use VS Code to:

- Open your cloned `61_math_flashcards` project
- View `main.py`
- View `game_engine.py`
- Make the required changes
- Run the Python game
- Use the VS Code terminal
- Check Git changes and commits

Your Git installation is already complete, and your repository is already cloned. '

Don't reinstall Git and don't clone the repository again.

Just install VS Code now and send me the **"Select Additional Tasks"** screen.

USER

done i installed vs code

ASSISTANT

Great! ' Now Git and VS Code are ready.

We already have your project cloned here:

TEXT

C:\Users\Admin\Desktop\Lab4\61_math_flashcards

STEP 7 — Open your cloned project in VS Code

We'll open **that exact folder**.

Method 1 — Using VS Code

1. Open **Visual Studio Code**.
2. At the top-left, click:

File !' Open Folder...

3. A Windows folder window will appear.
4. Navigate to:

TEXT

```
This PC
!' Local Disk (C:)
!' Users
!' Admin
!' Desktop
!' Lab4
!' 61_math_flashcards
```

5. Click the folder:

TEXT

61_math_flashcards

6. Click:

Select Folder

What you should see in VS Code

On the left side, you should see something similar to:

TEXT

```
61_MATH_FLASHCARDS
%
% % %   g a m e
%       % % %   g a m e _ e n g i n e . p y
%       % % %   t e x t _ b o x . p y
%
% % %   m a i n . p y
% % %   R E A D M E . m d
```

This is important because your README specifies this project structure.

& DON'T edit anything yet

Don't click inside the Python files and don't change anything.

Our next steps are:

Open project !' check Python interpreter !' install Pygame !' run ORIGINAL game !' reproduce the bug !' record BEFORE video.

The assignment requires the 10-second before video **before you make changes**.

Do this now

Open:

TEXT

C:\Users\Admin\Desktop\Lab4\61_math_flashcards

in VS Code.

Then **send me a screenshot of your VS Code window**, especially the left **Explorer** panel showing the files.

I'll guide you through the next step from there.

ASSISTANT

Perfect! ' You opened the **correct cloned repository** in VS Code.

I can see:

TEXT

```
61_math_flashcards
% % %   g a m e
%       % % %   g a m e _ e n g i n e . p y
%       % % %   t e x t _ b o x . p y
% % %   m a i n . p y
```

So everything is correct.

I also notice VS Code is showing **Restricted Mode** and a Python extension recommendation.

STEP 8 — Trust the project folder

At the top of VS Code you can see:

Restricted Mode is intended for safe code browsing. Trust this folder to enable all features.

Because this is the repository you intentionally cloned for your lab, we can trust it.

Do this:

1. Click **Manage** at the top.
2. A menu will appear.
3. Select:

Trust

or

Trust this folder & its files

4. If VS Code asks for confirmation, choose **Yes, I trust the authors**.

After that, the **Restricted Mode** message should disappear.

STEP 9 — Install the Python extension

At the bottom-right of your screenshot, VS Code is asking:

Do you want to install the recommended Python extension from Microsoft?

Click:

Install

This is important because we'll be running Python from VS Code.

Wait until the extension finishes installing.

Don't run the game yet

We need to do things in the correct order.

After the Python extension is installed, we'll:

1. Open the VS Code terminal.
2. Check Python.
3. Install **Pygame**.
4. Run the **original untouched game**.

5. Find the deliberate bug.
6. Record the required **10-second BEFORE video**.
7. Only then start Task 1.

Yourlab specifically says the before recording must happen **before making changes**.

For now, do only these two things:

\$Trust the folder

\$Click Install for the Microsoft Python extension

Then tell me “**done**” or send me a screenshot after both are finished.

USER

done

ASSISTANT

Great! ' Now the project is trusted and the Python extension is installed.

STEP 9 — Open the VS Code Terminal

We need to install **Pygame**, because your README says the game uses Pygame.

1. Open Terminal

In VS Code, look at the top menu.

Click:

Terminal !' New Terminal

A terminal panel should open at the **bottom** of VS Code.

You should see something similar to:

TEXT

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

or:

TEXT

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

2. Check Python

In that terminal, type:

BASH

```
python --version
```

Press **Enter**.

You should get:

TEXT

Python 3.13.15

3. Install Pygame

Now type:

BASH

```
python -m pip install pygame
```

Press **Enter**.

You should see Pygame being downloaded/installed.

At the end, you should see something similar to:

TEXT

Successfully installed pygame-...

& Important

Do not edit `game\_engine.py` or any other file.

We haven't made any changes yet because we need to record the original game first.

Do these two commands only:

BASH

```
python --version
```

then:

BASH

```
python -m pip install pygame
```

After they finish, **send me a screenshot of the terminal output.**

Then I'll show you how to run the **original game and reproduce the deliberate bug**, which is the next important step before recording your 10-second video.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> python --version Python 3.13.15 PS C:\Users\Admin\
```

ASSISTANT

Perfect. ' Python is working correctly inside your project.

Now install the only dependency your README specifies: **Pygame**.

In the same VS Code terminal, type:

POWERSHELL

```
python -m pip install pygame
```

Press **Enter** and wait for it to finish.

You should eventually see something like:

TEXT

```
Successfully installed pygame-...
```

Important

Don't run the game yet and don't change any Python files.

After the installation finishes, **send me the terminal output/screenshot**. Then I'll guide you through running the **original untouched game** and recording the required 10-second **BEFORE** video.

USER

```
done
```

ASSISTANT

Excellent. ' **Pygame is installed.**

Now we are at an important point: **we have not changed the original code**, so we can safely run it and make the required BEFORE recording.

STEP 10 — Run the original game

Your README says the game is started with:

BASH

```
python main.py
```

1. In the VS Code terminal

Make sure your terminal path ends with:

TEXT

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

Then type:

```
``powershell id="x0m9se" python main.py
```

Press ****Enter****.

```
### 2. What should happen?
```

A Pygame window should open showing your ****Math Flashcards**** game.

You should see something similar to:

- A math question/card
- An answer input box
- `SUBMIT` button
- Score
- Attempts
- Feedback/message area

STEP 11 — DO NOT FIX ANYTHING

This is the **original version**.

We need to demonstrate the deliberate bug for your BEFORE video.

Your README says the deliberate bug is:

> The mathematically correct answer is rejected because the operands are joined together as text instead of being calculated.

For example, if the game shows:

```
```text
3 + 5
```

the correct answer should be:

```
TEXT
8
```

But the broken calculation may effectively treat the operands as text, producing something like:

```
TEXT
35
```

We need to demonstrate whatever behavior **your actual original game shows**.

---

## STEP 12 — Test the original game

---

When the game opens:

**First, look at the question.**

For example:

```
TEXT
7 + 4
```

Calculate the correct answer yourself:

```
TEXT
11
```

Type:

TEXT

11

into the textbox.

Then press:

**Enter**

or click:

**SUBMIT**

### Watch what happens.

If the game says your correct answer is **incorrect**, that is the bug we need to show.

## STEP 13 — BEFORE VIDEO

Don't record yet until you've confirmed that the original game actually shows the bug.

Your assignment requires:

**A 10-second video of gameplay before your changes, showing the bug/broken behavior.**

So your video should ideally show:

**0–2 sec:** Game window visible with the question.

**2–6 sec:** You type the mathematically correct answer.

**6–10 sec:** You submit it and the game shows that the correct answer was rejected.

### Example

If it displays:

TEXT

12 + 7

you type:

TEXT

19

and submit.

If the game says:

TEXT

Incorrect!

Expected: 127

that is excellent evidence of the original bug.

**Do not deliberately change the code to create this behavior.** It should come from the original project.

---

## How to record on Windows

---

You can use Windows' built-in recording.

Press:

**Windows + Shift + R**

If your Windows version supports the Snipping Tool screen recording feature, a recording toolbar will appear.

Select the game window/area and record about **10 seconds**.

Alternatively, if your college normally uses another screen recorder, you can use that.

**Save it as:**

```
TEXT
before.mp4
```

Don't worry about creating the final `Lab-4` folder yet. We'll organize everything at the end.

---

**& Most important**

**Don't modify any code.**

First:

```
TEXT
python main.py
! "
Test original game
! "
Find the deliberate bug
! "
Record BEFORE video
```

After you've tested it, **tell me exactly what question appeared and what happened when you entered the correct answer.**

If possible, send me a screenshot of the game. Then I'll help you make the BEFORE recording correctly.

---

**ASSISTANT**

Perfect. ' I received your **BEFORE video**.

I checked the video file: it is about **9.7 seconds long**, which is essentially the required 10-second recording. Good. Ø=ÜM

So we can move forward.

---

## STEP 14 — Start Task 1: Fix the calculation bug

---

**Do not make the change yourself yet.** This lab specifically wants you to use an LLM as your debugging/pair-programming partner.

Your first task is:

## Fix the string-concatenation calculation bug.

We will do this properly:

**Inspect !' Prompt ChatGPT !' Understand suggestion !' Apply change !' Test !' Commit**

### First, let's inspect the relevant code

In VS Code, open:

TEXT

```
game
% % % g a m e _ e n g i n e . p y
```

You already have this file open.

Look at the code around:

PYTHON

```
def generate_new_card(self):
```

and also find the code that handles the answer when you press **Enter/SUBMIT**.

### Don't edit anything yet.

I want you to send me a screenshot of the **answer-checking part of `game\_engine.py`**.

You can use:

**Ctrl + F**

and search for:

TEXT

```
answer
```

or:

TEXT

```
submit
```

Then send me a screenshot showing that section.

After that, I'll give you the **exact Vibe Coding prompt to send to ChatGPT**, and we'll use the response to fix **Task 1** without skipping the LLM interaction your lab requires.

ASSISTANT

Excellent. ~~0=0~~ **This screenshot shows the exact deliberate bug.** We have found Task 1.

Look at your `game_engine.py`:

PYTHON

```
def compute_expected_answer(self):

 return int(f"{self.num_a}{self.num_b}")
```

## What is wrong?

This line:

PYTHON

```
int(f"{self.num_a}{self.num_b}")
```

joins the two numbers as **text**.

For example:

TEXT

```
num_a = 7
num_b = 5

f"{7}{5}"
! "
"75"
! "
int("75")
! "
75
```

But if the operator is **+**, the answer should be:

TEXT

```
7 + 5 = 12
```

So this is exactly the **string concatenation bug** described in your README.

## STEP 15 — Your Vibe Coding prompt for Task 1

Yourlab wants you to use an LLM as your debugging/pair-programming partner.

Since your VS Code has the **Chat/Agent** panel open on the right, let's use it.

### Click inside the Chat box

You currently have:

#### Describe what to build

Paste this prompt **exactly**:

TEXT

I am working on a Python Pygame Math Flashcards lab. Please analyze the answer calculation bug in game/game\_engine.py.

The method compute\_expected\_answer() currently returns:  
`int(f"{self.num_a}{self.num_b}")`

This incorrectly concatenates the operands as strings.

Fix Task 1 so that compute\_expected\_answer() correctly evaluates the arithmetic expression according to the active operator (+, -, \*).

Requirements:

- Keep the existing class structure and UI unchanged.
- Do not change unrelated code.
- Make the smallest clean fix possible.
- Explain what was wrong and what you changed.
- Make sure the returned answer is an integer.

## Why this is a good prompt

It tells the AI:

- **where** the problem is
- **what** the bug is
- **which operators** need to work
- **what must not be changed**
- **what explanation you need**

This is exactly the kind of iterative LLM interaction your assignment asks you to document.

## & VERY IMPORTANT

When the AI gives you a suggested change, **don't blindly accept it.**

We need to review what it suggests.

For example, a sensible solution would make the method behave like:

TEXT

```
+ !' n u m _ a + n u m _ b
- !' n u m _ a - n u m _ b
* !' n u m _ a * n u m _ b
```

But **wait for the AI's response first.**

## Do NOT click "Accept" / apply changes yet.

Send me a screenshot of the **AI's response in the VS Code Chat panel.**

I'll check whether its solution is correct before you apply it.

This is important because your lab is specifically asking you to **analyze, interact with the AI assistant, and critically review the code**, rather than simply accept whatever the AI generates.

Once we verify it, I'll tell you exactly how to apply it, test Task 1, and make your **first Git commit.**

USER

i that compasory to doo that

ASSISTANT

Yes— **for your Lab 4, using an LLM is compulsory.** Your README explicitly says:

**"Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab."**

And the handout says to **write prompts, fix the broken code, and add each feature using prompts.**

But don't worry: **you don't have to use the Chat panel inside VS Code specifically.** You are already using ChatGPT here, so we can use **this conversation** as your LLM interaction.

### What I recommend

Since you're a beginner, let's do everything here:

**ChatGPT (this chat) !' you understand the suggestion !' you make the change in VS Code !' test !' Git commit**

That is actually easier than using the VS Code Chat panel.

And your README's final submission requires the **complete Chat/LLM history**, so we should keep all our prompts and responses in this conversation.

### So you can ignore the VS Code Chat panel

You **do not need to click that Chat/Agent box.**

We'll continue from the code you showed me.

**Next:** I'll give you the exact Task 1 prompt/solution, explain every line in simple terms, then tell you exactly what to change in `game_engine.py`.

USER

wt i want to do now

ASSISTANT

Now you should **fix Task 1**. You have already completed all the setup:

' Python installed ' Git installed ' Repository cloned ' VS Code installed ' Project opened ' Python extension installed ' Pygame installed ' Original game run ' BEFORE video recorded ' Bug located in `game_engine.py`

So **don't reinstall anything and don't record again.**

## STEP 16 — Use ChatGPT for Task 1

Because your lab requires LLM interaction, we will use **this ChatGPT conversation** as your debugging/pair-programming record.

You found this broken code:

PYTHON

```
def compute_expected_answer(self):

 return int(f"{self.num_a}{self.num_b}")
```

### 1. Ask ChatGPT this prompt

You can consider the following prompt as your documented LLM prompt for Task 1:

TEXT

I am working on a Python Pygame Math Flashcards lab.

In `game/game_engine.py`, the method `compute_expected_answer()` currently uses:

```
return int(f"{self.num_a}{self.num_b}")
```

This incorrectly concatenates the two operands as strings.

Fix this bug so the game correctly calculates the answer based on the active operator (+, -, \*).

Keep the existing UI and class structure unchanged.

Make the smallest clean change possible.

Explain the change.

We have already identified the problem together, so **you don't need to send this to the VS Code Chat panel.**

## STEP 17 — Change the code

In VS Code, go to:

TEXT

```
game
!' g a m e _ e n g i n e . p y
```

Find:

PYTHON

```
def compute_expected_answer(self):

 return int(f"{self.num_a}{self.num_b}")
```

Replace **only that method** with:

PYTHON

```
def compute_expected_answer(self):
 if self.operator == "+":
 return self.num_a + self.num_b
 elif self.operator == "-":
 return self.num_a - self.num_b
 elif self.operator == "*":
 return self.num_a * self.num_b
```

### What this does

If the question is:

TEXT

```
7 + 5
```

it calculates:

TEXT

```
7 + 5 = 12
```

If it is:

TEXT

10 - 4

it calculates:

TEXT

10 - 4 = 6

If it is:

TEXT

6 \* 3

it calculates:

TEXT

6 \* 3 = 18

Instead of joining the numbers together.

---

## STEP 18 — Save the file

Press:

**Ctrl + S**

That's all for the code change.

**& Don't start Tasks 2, 3 or 4 yet.**

First we need to **test Task 1**.

After saving, tell me:

**`done`**

Then I'll show you exactly how to run the game again and prove that Task 1 is fixed.

---

USER

is that necceary to use the build with agent ??

ASSISTANT

No. **You do NOT need to use "Build with Agent."** L

Your lab says you must use an **LLM such as ChatGPT or Claude** as your debugging/pair-programming partner. It does **not** say you must use the VS Code **Build with Agent** feature. The handout says to use a Vibe Coding tool and write prompts to fix/add the tasks.

**For your assignment, this is enough:**

**You:** give the coding problem to ChatGPT !  
**ChatGPT:** suggests/explains the solution !  
**You:** review it and apply it in VS Code !  
**You:** test the program !  
**Git:** make a separate commit for the task

We are already doing that in **this ChatGPT conversation**, so you can ignore the **Build with Agent** panel.

## So right now

Go to `game_engine.py` and fix **Task 1** as I showed you:

PYTHON

```
def compute_expected_answer(self):
 if self.operator == "+":
 return self.num_a + self.num_b
 elif self.operator == "-":
 return self.num_a - self.num_b
 elif self.operator == "*":
 return self.num_a * self.num_b
```

Then press **Ctrl + S**.

**Don't do anything else yet.** Tell me when you've saved it, and I'll guide you through testing Task 1 and making the first separate Git commit.

USER

is this correct

ASSISTANT

Yes ~~0=~~ **the Task 1 code you entered is correct.** Your `compute_expected_answer()` now checks the operator and performs the actual arithmetic.

You have:

PYTHON

```
def compute_expected_answer(self):
 if self.operator == "+":
 return self.num_a + self.num_b
 elif self.operator == "-":
 return self.num_a - self.num_b
 elif self.operator == "*":
 return self.num_a * self.num_b
```

That correctly fixes the original concatenation bug.

## One important thing I notice

Your VS Code tab shows:

TEXT

```
game_engine.py 9 + %I
```

The **white dot** means the file has unsaved changes.

## Do this now

### 1. Save the file:

Press:

TEXT

Ctrl + S

The white dot should disappear.

## 2. Stop the old game process.

Your terminal still shows:

TEXT

```
python main.py
pygame 2.6.1 ...
Hello from the pygame community...
```

There isn't a new PowerShell prompt, which means the game process is still running.

Click inside the terminal and press:

TEXT

Ctrl + C

You should get your prompt back:

TEXT

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

## Then stop.

Don't run the game again yet.

After you've **saved the file and stopped the old process**, tell me **“done”**.

Then I'll show you exactly how to test Task 1 and verify that the bug is fixed before we make the first Git commit.

USER

done

ASSISTANT

Great. ' Task 1 is saved, and the old game is stopped.

## STEP 19 — Test Task 1

Now we need to prove that your fix actually works.

### 1. Run the game again

In the VS Code terminal, type:

```
``powershell id="t9v1a4" python main.py
```

Press **\*\*Enter\*\***.

The Math Flashcards game should open.

---

### 2. Look at the question

Suppose you get:

``text  
7 + 4

The correct answer is:

TEXT

11

Type **11** into the answer box and press **Enter** or click **SUBMIT**.

You should now get a **correct-answer response** and your score should increase.

### 3. Test the other operators too

Because Task 1 requires the active operator to work correctly, don't test only **+**.

Try:

#### Addition

TEXT

5 + 3 = 8

#### Subtraction

TEXT

9 - 4 = 5

#### Multiplication

TEXT

6 \* 3 = 18

You don't necessarily need those exact questions—the game generates random numbers. Just calculate whatever appears.

### What we're checking

Before your change:

TEXT

7 + 4

Answer: 11

'L I n c o r r e c t

After your change:

TEXT

7 + 4

Answer: 11

' C o r r e c t

That's the evidence that **Task 1 is fixed**.

## & Don't start Task 2 yet

Once you have confirmed that:

- `+` works
- `-` works
- `*` works

stop the game with:

TEXT

ctrl + c

Then tell me:

**"Task 1 works."**

After that, I'll guide you through the **Git commit for Task 1**. This is important because your lab specifically requires **separate commits for each task**.

USER

is that i want to record the screen

ASSISTANT

Yes— **you already recorded the BEFORE video**, so you do **not** need to record another before video. Your uploaded `before.mp4.mp4` is the before recording.

For the **AFTER video**, you will record after we finish **all 4 tasks**, because your README says the after video should show the bug fixed **and the new features working**.

**So the recording plan is:**

**Already done:**

- `<BEFORE video !'` original broken game

**Later:**

- `<AFTER video !'` Task 1 fixed + Timer + Streak + Division

Your lab handout requires both a before and after 10-second gameplay video.

**Right now**

Don't record anything.

Run:

POWERSHELL

```
python main.py
```

Test `+`, `-`, and `*`.

Once you're satisfied that Task 1 works, stop the game with:

TEXT

```
ctrl + c
```

Then tell me “**Task 1 works**”.

We'll make the **Task 1 Git commit**, and then move to **Task 2 — the timer bar**.

ASSISTANT

I see the problem. 'L**Don't record anything yet**. The game didn't start because there is an indentation error in the code we just changed.

The error says:

TEXT

```
IndentationError: expected an indented block after function definition on line 40
```

In your screenshot, line 41:

PYTHON

```
if self.operator == "+":
```

is at the **same indentation level as the function**, but it needs to be indented inside the function.

### Fix it like this

In `game_engine.py`, make sure the section looks **exactly like this**:

PYTHON

```
def compute_expected_answer(self):
 if self.operator == "+":
 return self.num_a + self.num_b
 elif self.operator == "-":
 return self.num_a - self.num_b
 elif self.operator == "*":
 return self.num_a * self.num_b
```

Notice the spaces:

TEXT

```
def compute_expected_answer(self):
 !" 4 s p a c e s
 if self.operator == "+":
 !" 8 s p a c e s
 return ...
```

### Easiest way to fix it

1. Select lines 40–46.

2. Delete them.

3. Paste this:

PYTHON

```
def compute_expected_answer(self):
 if self.operator == "+":
 return self.num_a + self.num_b
 elif self.operator == "-":
 return self.num_a - self.num_b
 elif self.operator == "*":
 return self.num_a * self.num_b
```

4. Press **Ctrl + S**.

Then run:

POWERSHELL

```
python main.py
```

## Important

This is exactly why we test the AI-generated/code change instead of blindly assuming it works. Your screenshot shows the **logic was right, but the indentation was wrong**.

If the game opens successfully, **don't record yet**. First test one **+**, one **-**, and one **\*** question.

Send me a screenshot after you run it.

ASSISTANT

Yes! ~~0<10~~ **The game is running now**, so the indentation problem is fixed.

And your screenshot shows:

TEXT

```
11 + 11
```

and:

TEXT

```
CORRECT!
```

So **Task 1 is working**.

One thing I notice: the feedback says **CORRECT! 13 - 6 = 7**, which appears to be the previous question, while the current question is **11 + 11**. That's likely just the game's existing feedback behavior, not necessarily a Task 1 problem.

## Do this now — test Task 1 properly

### Test 1: Addition

Current question:

TEXT

```
11 + 11
```

Type:

TEXT

22

Click **SUBMIT**.

You should get a correct result and the score should increase.

## Test 2: Subtraction

Wait for a question containing  $-$ .

For example:

TEXT

13 - 6

Type the correct answer:

TEXT

7

Submit it.

## Test 3: Multiplication

Wait for a question containing  $*$ .

For example:

TEXT

5 \* 4

Type:

TEXT

20

Submit it.

If all three work, **Task 1 is successfully tested.**

## Don't record the AFTER video yet.

We still have:

- Task 2 — Timer  $\neq$
- Task 3 — Streak multiplier  $\neq$
- Task 4 — Division  $\neq$

The AFTER video comes **after all four are completed**.

After you've tested  $+$ ,  $-$ , and  $*$ , stop the game with:

TEXT

ctrl + c

Then tell me “**Task 1 tested**”.

Next I'll guide you through the **separate Git commit for Task 1**.

USER

when it stop i keep gonig and now my score is 15/15

ASSISTANT

That's actually good!  $\emptyset=15/15$  means the game is running and you're answering correctly.

You **don't need to stop the game immediately** just because you reached 15/15. I wanted you to stop it after testing the operators.

**Before stopping, confirm these 3 things:**

- ' You answered an **addition (+)** question correctly.
- ' You answered a **subtraction (-)** question correctly.
- ' You answered a *\*multiplication (\*)* question correctly.
- ' Score increased for correct answers.

If you've already done all three, **Task 1 is tested successfully**.  $\emptyset<100\%$

**Now stop the game**

Click the **VS Code terminal** behind the game, then press:

TEXT

Ctrl + c

The Pygame window should close, and your terminal should return to:

TEXT

PS C:\Users\Admin\Desktop\Lab4\61\_math\_flashcards>

**Then tell me:**

**Task 1 tested**

After that, we'll make the **Task 1 Git commit**. Remember, your assignment requires a **separate commit for each task**.

USER

done

ASSISTANT

Perfect. ' **Task 1 is tested successfully**.

Now we need to save Task 1 properly in Git before starting Task 2.

## STEP 20 — Make the Task 1 Git commit

A **commit** is basically a saved checkpoint in Git.

Your teacher specifically requires:

**Separate commits for each task.**

So we'll make one commit called something like:

TEXT

Fix arithmetic calculation bug

### 1. Make sure the game is stopped

Your VS Code terminal should show:

TEXT

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

### 2. Check what Git detected

Type:

```
```powershell id="4f2w2c" git status
```

Press ****Enter****.

You should see something indicating that:

```
```text
game/game_engine.py
```

was modified.

### 3. Add only the Task 1 file

Type:

```
```powershell id="a0ghc4" git add game/game_engine.py
```

Press ****Enter****.

4. Create the Task 1 commit

Type:

```
```powershell id="8p6u9v"
git commit -m "Fix arithmetic calculation bug"
```

Press **Enter**.

You should see something similar to:

TEXT

```
[main xxxxxxx] Fix arithmetic calculation bug
1 file changed
...
```

0<1% That means **Task 1 has its own Git commit.**

## & Don't push yet

We are **not pushing to GitHub yet.**

We still need to complete Tasks 2, 3 and 4.

Our commits will eventually look approximately like:

TEXT

```
Fix arithmetic calculation bug
Add per-question timer
Add streak score multiplier
Add integer division
```

Then we'll push everything to **your own repository** at the end.

**Do these commands now, one at a time:**

```
```powershell id="2bgm8p" git status
```

then:

```
```powershell id="7w2qgk"
git add game/game_engine.py
```

then:

```
```powershell id="vwr5y4" git commit -m "Fix arithmetic calculation bug"
```

****Send me the output of the `git commit` command.** Then we'll start **Task 2: the countdown timer bar**.**

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
Changes not staged for commit: (use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
modified:   game/game_engine.py
```

```
Untracked files: (use "git add <file>..." to include in what will be committed)
game/pycache/
```

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect. `git status` shows exactly what we expected:

- `game/game_engine.py` !' **modified** ' — this is Task 1.
- `game/__pycache__` / !' **untracked** & b — this is automatically generated by Python and **should NOT be committed**.

So we should commit **only** `game_engine.py`.

STEP 20A — Add only Task 1

Run:

POWERSHELL

```
git add game/game_engine.py
```

Then check:

POWERSHELL

```
git status
```

You should see `game_engine.py` under **Changes to be committed**.

The `__pycache__` folder may still appear under **Untracked files**. That's okay—we won't add it.

STEP 20B — Create the Task 1 commit

Run:

POWERSHELL

```
git commit -m "Fix arithmetic calculation bug"
```

You should get something like:

TEXT

```
[main abc1234] Fix arithmetic calculation bug
1 file changed
```

That means **Task 1 has its own commit**.

Don't push yet.

Send me the output after running the `git commit` command. Then we'll start **Task 2: the per-question countdown timer**.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git add game/game_engine.py PS
```

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status On branch main Your branch is up to date with 'origin/main'.
```

```
Changes to be committed: (use "git restore --staged <file>..." to unstage) modified:   game/game_engine.py
```

```
Untracked files: (use "git add <file>..." to include in what will be committed) game/__pycache/
```

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git commit -m "Fix arithmetic calculation bug" Author identity unknown
```

*** Please tell me who you are.

Run

```
git config --global user.email "you@example.com" git config --global user.name "Your Name"
```

to set your account's default identity. Omit --global to set the identity only in this repository.

```
fatal: unable to auto-detect email address (got 'Admin@DESKTOP-5SR1F9S.(none)') PS
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

That's completely normal. ~~Nothing is wrong with your project.~~ Git just doesn't know your name and email yet. Your Task 1 change is still safely staged. **Do not run `git add` again.**

STEP 20C — Set your Git name and email

We need to tell Git who is making the commits.

First, set your name

Use your name:

POWERSHELL

```
git config --global user.name "Purvik Gowda"
```

Press **Enter**.

Then set your email

Here you should use the **email associated with your GitHub account**.

For example, if your GitHub email is:

TEXT

```
yourname@gmail.com
```

run:

POWERSHELL

```
git config --global user.email "yourname@gmail.com"
```

& **Don't use the example `you@example.com`**. Use the email you actually use for GitHub.

Then verify

Run:

POWERSHELL

```
git config --global user.name
```

and:

POWERSHELL

```
git config --global user.email
```

You should see your name and your GitHub email.

Finally, make the commit again

POWERSHELL

```
git commit -m "Fix arithmetic calculation bug"
```

This time it should work.

Important

You **do not need to configure Git again for every task**. Because we used `--global`, Git will remember your name and email for future commits on this computer.

Also, your `game/__pycache__` is still untracked, which is fine. **Don't add it.**

Run the commands above and send me the output of the final `git commit`.

USER

my gmail is purvikgowda33@gmail.com

ASSISTANT

Perfect. ~~Use~~ Use that email for your Git configuration.

In the VS Code terminal, run these **three commands one by one**:

1. Set your Git name

```
``powershell id="6n8m2s" git config --global user.name "Purvik Gowda"
```

2. Set your GitHub email

```
```powershell id="6g4w5x"
git config --global user.email "purvikgowda33@gmail.com"
```

### 3. Make the Task 1 commit again

Your `game_engine.py` is **already staged**, so you don't need `git add` again.

Run:

```
``powershell id="3l8f4q" git commit -m "Fix arithmetic calculation bug"
```

You should get something similar to:

```
```text
[main abc1234] Fix arithmetic calculation bug
1 file changed, ...
```

' That means **Task 1 is officially committed**.

Don't push yet.

Send me the output of the `git commit` command, and then we'll begin **Task 2 — adding the countdown timer bar**.

USER

done

ASSISTANT

Excellent. ' **Task 1 is now committed.**

Your Git history should now contain:

TEXT

Fix arithmetic calculation bug

We can move to **Task 2**.

STEP 21 — Task 2: Add the Per-Question Timer

Your README says:

Players currently have unlimited time to answer each flashcard. Add a visible countdown timer bar below the active card that tracks remaining time and automatically registers a missed attempt if time runs out before submission.

So Task 2 needs **three things**:

1. `#if` A countdown for every question.
2. `0=0` A visible timer bar below the card.
3. 'L When time reaches zero, automatically count it as a missed attempt and move/handle the question appropriately.

Before we change anything

We should use our LLM process again:

Prompt !' understand the suggestion !' apply !' test !' commit

And we'll make **another separate Git commit** for Task 2.

STEP 22 — First inspect the existing code

Open:

TEXT

game/game_engine.py

We need to understand how the game currently:

- creates a new question
- handles time/events
- submits an answer
- counts attempts

- moves to the next question

Don't edit anything yet.

In VS Code press:

TEXT

Ctrl + F

Search for:

TEXT

handle_event

If you don't find it, search:

TEXT

submit

or:

TEXT

attempt

Send me a screenshot

I want to see the relevant section of `game_engine.py` where the game processes keyboard/button input and submits the answer.

Don't change the code yet.

Once I see that section, I'll give you a proper Task 2 prompt and we'll implement the timer without accidentally breaking the existing game.

ASSISTANT

Perfect! I can see the relevant part of `game_engine.py`.

Your **Task 1 commit is successful**, so now we can start **Task 2: Per-question timer**.

STEP 21 — Do this now

In your `game_engine.py`, you can see:

PYTHON

```
def update(self):  
    pass
```

This is exactly the place where the timer logic can be added.

Do not edit anything yet.

1. Scroll to the top of `game_engine.py`

Scroll all the way up so we can see:

- `__init__()`

- `generate_new_card()`
- the variables created when the game starts.

Take a screenshot showing roughly **lines 1–45** and send it here.

I need to see those variables before I give you the Task 2 prompt, because the timer needs to connect correctly with the existing game structure.

Only do this now: scroll to the top !' take screenshot !' send it.

ASSISTANT

I received the screen recording ~~Ø=Ü~~ checked it, and it shows the top portion of `game_engine.py`, including the `__init__()` variables and `generate_new_card()`.

So you **do not need to send another screenshot**.

STEP 22 — Task 2 prompt

Now we will use this prompt in the ChatGPT conversation for the lab record.

Copy and send this as your next message to me:

I am working on Task 2 of my Python Pygame Math Flashcards lab. I need to add a per-question countdown timer bar. Please modify my existing `game/game_engine.py` with the smallest clean changes possible. The timer should:

- 1. Show a visible countdown/progress bar below the active flashcard.**
- 2. Give each question a fixed amount of time.**
- 3. Decrease continuously while the question is active.**
- 4. Automatically register a missed attempt when the timer reaches zero.**
- 5. Reset the timer when a new question is generated.**
- 6. Reset the timer after a valid answer moves to the next question.**
- 7. Reset the streak/score appropriately on timeout if required by the existing game logic.**
- 8. Keep the existing UI, input box, buttons, and arithmetic functionality working.**
- 9. Do not remove or unnecessarily rewrite existing code.**

Please first explain exactly which parts of `game_engine.py` need to be changed and then provide the complete updated code.

Do not change your code yet.

Send that prompt to me, and I will give you the exact changes **one step at a time**.

USER

Task 2

ASSISTANT

Yes ~~Ø=Ü~~ **Task 2 is the Timer Task.**

We will do it carefully and **one step at a time**.

STEP 22 — Send this prompt to me

Copy and paste this exact prompt:

I am working on Task 2 of my Python Pygame Math Flashcards lab. I need to add a per-question countdown timer bar. Please modify my existing `game/game_engine.py` with the smallest clean changes possible. The timer should:

1. Show a visible countdown/progress bar below the active flashcard. 2. Give each question a fixed amount of time. 3. Decrease continuously while the question is active. 4. Automatically register a missed attempt when the timer reaches zero. 5. Reset the timer when a new question is generated. 6. Reset the timer after a valid answer moves to the next question. 7. Reset the streak/score appropriately on timeout if required by the existing game logic. 8. Keep the existing UI, input box, buttons, and arithmetic functionality working. 9. Do not remove or unnecessarily rewrite existing code.

First explain which parts of `game_engine.py` need to be changed, then provide the updated code.

Don't edit the code yet.

Send that prompt, and after you get the response, **send me the response/screenshot here**. I will tell you exactly what to do next.

USER

where i want to paste this]

ASSISTANT

Don't paste it into `game_engine.py` Ø=ÜM

Since you're using **this ChatGPT conversation as your LLM/Vibe Coding tool**, paste the prompt **here in the ChatGPT message box**.

Do this now

1. Stay in this ChatGPT conversation.
2. Click the **message box at the bottom**.
3. Paste the entire **Task 2 prompt** I gave you.
4. Press **Enter**.
5. I will give you the changes needed for `game_engine.py`.
6. **Do not copy anything into VS Code yet** until I tell you exactly what to change.

This conversation itself is also useful later because your lab requires the **LLM/chat history** as part of the submission.

So for now: **paste the Task 2 prompt right here in ChatGPT**.

USER

is this neccesary

ASSISTANT

No. **You don't need to paste that prompt anywhere else**. Ø=ÜM

I gave you that prompt only to help you document the "Vibe Coding" interaction, but since we're already doing the work together in this ChatGPT conversation, **we can continue directly here**.

For the lab, the important part is that you:

- use an LLM to fix/add the features,
- keep the chat history,
- make a separate Git commit for each task,
- and complete the README requirements.

So let's continue directly.

Do not change any code yet.

I already have the relevant `game_engine.py` structure from your screenshot. I'll give you the **exact Task 2 code changes one step at a time**.

If you're ready, I'll give you **STEP 22 — adding the timer variables**.

USER

continue

ASSISTANT

Yes, I can continue **Task 2** directly.

STEP 22 — Add the timer variables

We will first add only the timer variables. **Don't change anything else.**

1. Open `game/game_engine.py` in VS Code.
2. Scroll to the `__init__(self)` section near the top.
3. Find the existing variables there.
4. **Send me a screenshot of lines around `__init__()`** (approximately lines 1–45).

I want to make sure I give you the exact location based on your actual code.

Do not type the timer code yet. Send the screenshot first.

ASSISTANT

Perfect. I can clearly see your `__init__()` and `generate_new_card()` sections now.

We can add the timer safely.

STEP 23 — Add timer variables

In `game_engine.py`, inside `__init__()`, find this line:

PYTHON

```
self.feedback_color = (200, 205, 215)
```

Immediately **below it**, add these 3 lines:

PYTHON

```
self.time_limit = 10
self.time_remaining = self.time_limit
self.timer_start = pygame.time.get_ticks()
```

So it should look like:

PYTHON

```
self.score = 0
self.total_attempts = 0
self.feedback_msg = "Solve the card and press Enter!"
self.feedback_color = (200, 205, 215)

self.time_limit = 10
self.time_remaining = self.time_limit
self.timer_start = pygame.time.get_ticks()
```

Important

- Don't change any other lines.
- Don't add anything to `generate_new_card()` yet.
- Don't run the program yet.

After adding these **3 lines**, tell me **DONE**. Then I'll give you the next step.

USER

```
self.score = 0 self.total_attempts = 0 self.feedback_msg = "Solve the card and press Enter!" self.feedback_color =
(200, 205, 215)

self.time_limit = 10 self.time_remaining = self.time_limit self.timer_start = pygame.time.get_ticks()

self.num_a = 0 self.num_b = 0 self.operator = "+" done
```

ASSISTANT

Perfect ' The 3 timer variables are in the correct place.

STEP 24 — Reset the timer for every new question

Now we need to make sure that **every time a new flashcard appears, the 10-second timer starts again**.

In your `generate_new_card()` function, you currently have:

PYTHON

```
self.input_box.clear()
```

Add these **3 lines immediately below it**:

PYTHON

```
self.time_remaining = self.time_limit
self.timer_start = pygame.time.get_ticks()
```

So the bottom of `generate_new_card()` should look like:

PYTHON

```
if self.operator == "-" and self.num_a < self.num_b:
    self.num_a, self.num_b = self.num_b, self.num_a

self.input_box.clear()
```

```
self.time_remaining = self.time_limit
self.timer_start = pygame.time.get_ticks()
```

Don't change anything else.

When you finish, reply **DONE**.

USER

done

ASSISTANT

Great '

STEP 25 — Update the `update()` function

Right now your code has:

PYTHON

```
def update(self):
    pass
```

We will replace **only** `pass` with the timer logic.

Change it to:

PYTHON

```
def update(self):
    elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000
    self.time_remaining = max(0, self.time_limit - elapsed)

    if self.time_remaining <= 0:
        self.total_attempts += 1
        self.feedback_msg = "TIME UP! Try the next card."
        self.feedback_color = (240, 180, 70)
        self.generate_new_card()
```

Important

Your function should now look **exactly like this**:

PYTHON

```
def update(self):
    elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000
    self.time_remaining = max(0, self.time_limit - elapsed)

    if self.time_remaining <= 0:
        self.total_attempts += 1
        self.feedback_msg = "TIME UP! Try the next card."
        self.feedback_color = (240, 180, 70)
        self.generate_new_card()
```

Do not run the game yet.

After you replace `pass` with this code, reply **DONE**.

USER

i think best job is give full modified code. is that okay for my project ??

ASSISTANT

Yes ~~0=0~~ **Giving you the full modified `game\_engine.py` code is okay**, and for your project it can actually be easier and safer than changing many small pieces manually.

But there is one important point for your lab: the handout specifically wants you to **use prompts to fix/add features one by one and make a separate commit for each task**.

So we can still do it correctly:

- **Task 1:** already completed and committed '
- **Task 2:** I'll give you the **complete modified `game\_engine.py`** for the timer.
- You replace the file with that version.
- We test it.
- If it works, make **one Task 2 commit**.
- Then Task 3 and Task 4 separately.

This is better than making you manually edit 10–15 places and accidentally causing indentation errors.

However, I need the complete current `game\_engine.py` before giving you the full replacement, because I don't want to accidentally remove code that isn't visible in the screenshots.

Next step

Please **copy the entire contents of `game\_engine.py`** from VS Code and paste it here.

In VS Code:

Ctrl + A !' Ctrl + C !' come here !' Ctrl + V

Then I'll return the **complete Task 2 modified code**, keeping your Task 1 fix intact.

USER

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator =
```

```

random.choice(["+", "-", "*"]) if self.operator == "-" and self.num_a < self.num_b: self.num_a, self.num_b =
self.num_b, self.num_a

self.input_box.clear()

self.time_remaining = self.time_limit self.timer_start = pygame.time.get_ticks()

def compute_expected_answer(self): if self.operator == "+": return self.num_a + self.num_b elif self.operator == "-":
return self.num_a - self.num_b elif self.operator == "*": return self.num_a * self.num_b

def submit_answer(self): val_str = self.input_box.text.strip() if not val_str or val_str == "-": self.feedback_msg = "Type
an answer first!" self.feedback_color = (240, 175, 40) return

user_answer = int(val_str) expected = self.compute_expected_answer() self.total_attempts += 1

if user_answer == expected: self.score += 1 self.feedback_msg = f"CORRECT! {self.num_a} {self.operator}
{self.num_b} = {expected}" self.feedback_color = (80, 230, 110) self.generate_new_card() else: self.feedback_msg =
f"WRONG! Expected {expected}." self.feedback_color = (240, 75, 75) self.input_box.clear()

def handle_event(self, event): self.input_box.handle_event(event)

if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN: self.submit_answer() elif event.type ==
pygame.MOUSEBUTTONDOWN and event.button == 1: if self.submit_btn.collidepoint(event.pos):
self.submit_answer()

def update(self): pass

def render(self, screen): screen.fill((25, 29, 37))

title_surf = self.font_title.render("Math Flashcards Arena", True, (245, 245, 245)) screen.blit(title_surf, (self.width // 2 -
title_surf.get_width() // 2, 18))

score_surf = self.font_hud.render(f"Score: {self.score} / {self.total_attempts}", True, (255, 220, 80))
screen.blit(score_surf, (self.width // 2 - score_surf.get_width() // 2, 58))

card_rect = pygame.Rect(self.width // 2 - 130, 95, 260, 110) pygame.draw.rect(screen, (240, 242, 245), card_rect,
border_radius=12) pygame.draw.rect(screen, (85, 120, 175), card_rect, width=3, border_radius=12)

card_str = f"{self.num_a} {self.operator} {self.num_b}" card_surf = self.font_card.render(card_str, True, (25, 30, 42))
screen.blit(card_surf, (card_rect.centerx - card_surf.get_width() // 2, card_rect.centery - card_surf.get_height() // 2))

self.input_box.render(screen)

pygame.draw.rect(screen, (45, 140, 80), self.submit_btn, border_radius=6) pygame.draw.rect(screen, (215, 225,
220), self.submit_btn, width=2, border_radius=6) btn_txt = self.font_btn.render("SUBMIT", True, (255, 255, 255))
screen.blit(btn_txt, (self.submit_btn.centerx - btn_txt.get_width() // 2, self.submit_btn.centery - btn_txt.get_height() //
2))

msg_surf = self.font_hud.render(self.feedback_msg, True, self.feedback_color) screen.blit(msg_surf, (self.width // 2 -
msg_surf.get_width() // 2, 295))

```

ASSISTANT

Yes ~~0=0~~ Since you gave me the **complete current `game\_engine.py`**, I can safely give you the full Task 2 version.

This version:

- Keeps your **Task 1 arithmetic fix** '
- Adds a **10-second timer** '
- Resets timer for every new card '
- Counts a timeout as an attempt '
- Automatically moves to the next card when time reaches zero '

- Shows a **visible timer/progress bar below the question** '
- Does **not** add Task 3 or Task 4 yet.

STEP 26 — Replace the complete `game_engine.py`

In VS Code:

1. Open `game_engine.py`.
2. Press **Ctrl + A**.
3. Press **Backspace**.
4. Paste the complete code below.
5. Press **Ctrl + S**.

PYTHON

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator = random.choice(["+", "-", "*"])

        if self.operator == "-" and self.num_a < self.num_b:
            self.num_a, self.num_b = self.num_b, self.num_a

        self.input_box.clear()

        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

    def compute_expected_answer(self):
```

```

        if self.operator == "+":
            return self.num_a + self.num_b
        elif self.operator == "-":
            return self.num_a - self.num_b
        elif self.operator == "*":
            return self.num_a * self.num_b

def submit_answer(self):
    val_str = self.input_box.text.strip()

    if not val_str or val_str == "-":
        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return

    user_answer = int(val_str)
    expected = self.compute_expected_answer()
    self.total_attempts += 1

    if user_answer == expected:
        self.score += 1
        self.feedback_msg = (
            f"CORRECT! {self.num_a} {self.operator} "
            f"{self.num_b} = {expected}"
        )
        self.feedback_color = (80, 230, 110)
        self.generate_new_card()
    else:
        self.feedback_msg = f"WRONG! Expected {expected}."
        self.feedback_color = (240, 75, 75)
        self.input_box.clear()

def handle_event(self, event):
    self.input_box.handle_event(event)

    if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
        self.submit_answer()

    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if self.submit_btn.collidepoint(event.pos):
            self.submit_answer()

def update(self):
    elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000
    self.time_remaining = max(0, self.time_limit - elapsed)

    if self.time_remaining <= 0:
        self.total_attempts += 1
        self.feedback_msg = "TIME UP! Try the next card."
        self.feedback_color = (240, 180, 70)
        self.generate_new_card()

def render(self, screen):
    screen.fill((25, 29, 37))

    title_surf = self.font_title.render(
        "Math Flashcards Arena",
        True,
        (245, 245, 245)
    )
    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            18
        )
    )

```

```

    )

    score_surf = self.font_hud.render(
        f"Score: {self.score} / {self.total_attempts}",
        True,
        (255, 220, 80)
    )
    screen.blit(
        score_surf,
        (
            self.width // 2 - score_surf.get_width() // 2,
            58
        )
    )

    card_rect = pygame.Rect(
        self.width // 2 - 130,
        95,
        260,
        110
    )

    pygame.draw.rect(
        screen,
        (240, 242, 245),
        card_rect,
        border_radius=12
    )

    pygame.draw.rect(
        screen,
        (85, 120, 175),
        card_rect,
        width=3,
        border_radius=12
    )

    card_str = f"{self.num_a} {self.operator} {self.num_b}"
    card_surf = self.font_card.render(
        card_str,
        True,
        (25, 30, 42)
    )

    screen.blit(
        card_surf,
        (
            card_rect.centerx - card_surf.get_width() // 2,
            card_rect.centery - card_surf.get_height() // 2
        )
    )

    # Timer bar
    timer_rect = pygame.Rect(
        self.width // 2 - 130,
        212,
        260,
        10
    )

    pygame.draw.rect(
        screen,
        (65, 70, 80),
        timer_rect,
        border_radius=5
    )

```

```

timer_width = int(
    timer_rect.width *
    (self.time_remaining / self.time_limit)
)

if timer_width > 0:
    pygame.draw.rect(
        screen,
        (80, 200, 110),
        pygame.Rect(
            timer_rect.x,
            timer_rect.y,
            timer_width,
            timer_rect.height
        ),
        border_radius=5
    )

timer_text = self.font_btn.render(
    f"Time: {self.time_remaining:.1f}s",
    True,
    (220, 225, 230)
)

screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        330
    )
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (45, 140, 80),
    self.submit_btn,
    border_radius=6
)

pygame.draw.rect(
    screen,
    (215, 225, 220),
    self.submit_btn,
    width=2,
    border_radius=6
)

btn_txt = self.font_btn.render(
    "SUBMIT",
    True,
    (255, 255, 255)
)

screen.blit(
    btn_txt,
    (
        self.submit_btn.centerx - btn_txt.get_width() // 2,
        self.submit_btn.centery - btn_txt.get_height() // 2
    )
)

msg_surf = self.font_hud.render(
    self.feedback_msg,

```

```
        True,
        self.feedback_color
    )

    screen.blit(
        msg_surf,
        (
            self.width // 2 - msg_surf.get_width() // 2,
            295
        )
    )
)
```

& One important thing

I noticed the timer text is at `y = 330`, while your feedback message is at `y = 295`. That means they can overlap.

Don't worry about fixing that now. First let's see the actual game. We can adjust the UI after testing if necessary.

For now, do ONLY this:

Ctrl + A !' delete !' paste the complete code !' Ctrl + S

Then **do not run it yet**.

Reply **DONE** when you have saved it.

USER

done

ASSISTANT

Great ' Task2 code is saved.

STEP 27 — Run and test Task 2

Now let's check whether the timer works.

In the VS Code terminal, make sure you are inside:

TEXT

61_math_flashcards

Then type:

BASH

python main.py

Press **Enter**.

What you should see

A game window should open.

Look below the math question. You should see:

- A **timer/progress bar**
- `Time: 10.0s` counting down

- The bar becoming shorter
- After about 10 seconds, the question should automatically change
- The **attempt count should increase**

Test only the timer first

Don't worry about correct/wrong answers yet.

Let **one question reach 0 seconds** and observe what happens.

Then close the game.

Tell me exactly what happened:

A. Timer works and question changes **B.** Timer appears but something overlaps/looks wrong **C.** Game gives an error
D. Timer doesn't appear

If there is an error, **send me a screenshot of the terminal.**

ASSISTANT

Excellent! **Task 2 is working.**

From your screenshot I can see:

- ' Timer bar is visible below the question.
- ' Timer is counting down (Time: 5.3s).
- ' The score/attempt counter changed to 15 / 16 , showing the timeout was counted as an attempt.
- ' The game is running without an error.

There is one small UI issue: the previous “**CORRECT!**” **message remains visible** after a new card appears. We can clean that up later if needed.

STEP 28 — Test the actual timeout

Before we commit Task2, let's verify the most important requirement.

1. Keep the game open.
2. **Do not type an answer.**
3. Wait until the timer reaches 0.0s .
4. Watch what happens.

You should see:

- TIME UP! Try the next card.
- The question changes.
- The attempt count increases by 1.
- The timer starts again near 10.0s .

After you see that, **close the game** and tell me what happened.

Don't commit yet.

USER

done

ASSISTANT

Perfect ' **Task 2 has been successfully tested.**

Now we need to make the required **separate Git commit for Task 2.**

STEP 29 — Check Git status

In the VS Code terminal, type:

```
BASH
git status
```

Press **Enter**.

You should see something similar to:

```
TEXT
modified:   game/game_engine.py
```

You may also see:

```
TEXT
__pycache__
```

& **Do not add `\_\_pycache\_\_` to Git.**

Send me a screenshot of the `git status` output before we commit.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit. (use "git push" to publish your local commits)
```

```
Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git restore <file>..." to discard changes in working directory)
modified:   game/game_engine.py
```

```
Untracked files: (use "git add <file>..." to include in what will be committed)
game/__pycache__/
```

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect! Your Git status is exactly what we expected.

- `game/game_engine.py` **needs to be committed**
- `game/__pycache__` **do NOT commit**
- Your Task 1 commit is already safe locally

STEP 30 — Stage only `game_engine.py`

In the same terminal, type:

BASH

```
git add game/game_engine.py
```

Press **Enter**.

Then type:

BASH

```
git status
```

Press **Enter**.

You should see:

TEXT

```
Changes to be committed:
  modified:   game/game_engine.py
```

And `__pycache__` may still appear under **Untracked files**. That's okay.

Don't commit yet.

Send me the new `git status` output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git add game/game_engine.py
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit. (use "git push" to publish your local commits)

Changes to be committed: (use "git restore --staged <file>..." to unstage)
  modified:   game/game_engine.py

Untracked files: (use "git add <file>..." to include in what will be committed)
  game/__pycache__/

PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect! **Task 2 is correctly staged.**

The important part is:

TEXT

```
Changes to be committed:
  modified:   game/game_engine.py
```

And `__pycache__` is still **not staged**, which is exactly what we want. Ø=ÜM

STEP 31 — Commit Task 2

Now type this exact command:

BASH

```
git commit -m "Add per-question timer"
```

Press **Enter**.

You should get something similar to:

TEXT

```
[main XXXXXXX] Add per-question timer  
1 file changed, ...
```

Do not run `git push` yet. We will push everything only after Tasks 3 and 4 are completed.

Send me the commit output after you run it.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git commit -m "Add per-question timer" [main 34957d4] Add  
per-question timer 1 file changed, 168 insertions(+), 17 deletions(-) PS  
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Excellent! **Task 2 is now committed successfully.**

Your Git history now has:

1. Fix arithmetic calculation bug '
2. Add per-question timer '

We are following the lab requirement of making a **separate commit for each task**.

Next: Task 3 — Streak Multipliers

Task 3 requires:

Consecutive correct answers should give increasing score multipliers, and the streak should reset after a wrong answer or timeout.

Don't change anything yet.

Before we modify the code, I'll guide you through it one step at a time just like Tasks 1 and 2.

Reply **READY** and we'll start Task 3.

USER

ready

ASSISTANT

Great! **Task 3 — Consecutive Correct Streak Multipliers.**

The README says the current game gives flat points for correct answers. We need to make consecutive correct answers increase the score, and reset the streak after a wrong answer or timeout.

STEP 32 — Add the streak variables

Open `game/game_engine.py`.

Find:

```
PYTHON
self.score = 0
self.total_attempts = 0
```

Immediately below them, add:

```
PYTHON
self.streak = 0
self.streak_multiplier = 1
```

So it becomes:

```
PYTHON
self.score = 0
self.total_attempts = 0
self.streak = 0
self.streak_multiplier = 1
```

Don't change anything else yet.

After adding those two lines, reply **DONE**.

```
USER
done
```

ASSISTANT

Perfect ' The streak variables are added correctly.

STEP 33 — Change the correct-answer scoring

Now find this part inside `submit_answer()`:

```
PYTHON
if user_answer == expected:
    self.score += 1
    self.feedback_msg = (
```

Change:

```
PYTHON
self.score += 1
```

to these **3 lines**:

PYTHON

```
self.streak += 1
self.streak_multiplier = min(self.streak, 3)
self.score += self.streak_multiplier
```

So that section should become:

PYTHON

```
if user_answer == expected:
    self.streak += 1
    self.streak_multiplier = min(self.streak, 3)
    self.score += self.streak_multiplier

    self.feedback_msg = (
        f"CORRECT! {self.num_a} {self.operator} "
        f"{self.num_b} = {expected}"
    )
```

What this means

- 1st correct !' **+1 point**
- 2nd consecutive correct !' **+2 points**
- 3rd consecutive correct !' **+3 points**
- 4th consecutive correct !' **+3 points**
- 5th consecutive correct !' **+3 points**

So the multiplier increases up to **3×**.

Don't change the wrong-answer part yet.

After making this change, reply **DONE**.

USER

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0
        self.streak = 0
        self.streak_multiplier = 1
        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator = random.choice(["+", "-", "*"])

        if self.operator == "-" and self.num_a < self.num_b:
            self.num_a, self.num_b = self.num_b, self.num_a

        self.input_box.clear()
```

```

self.time_remaining = self.time_limit self.timer_start = pygame.time.get_ticks()

def compute_expected_answer(self): if self.operator == "+": return self.num_a + self.num_b elif self.operator == "-":
return self.num_a - self.num_b elif self.operator == "": return self.num_a self.num_b

def submit_answer(self): val_str = self.input_box.text.strip()

if not val_str or val_str == "-": self.feedback_msg = "Type an answer first!" self.feedback_color = (240, 175, 40) return
user_answer = int(val_str) expected = self.compute_expected_answer() self.total_attempts += 1

if user_answer == expected: self.score += 1 self.feedback_msg = ( f"CORRECT! {self.num_a} {self.operator} "
f"{self.num_b} = {expected}" ) self.feedback_color = (80, 230, 110) self.generate_new_card() else:
self.feedback_msg = f"WRONG! Expected {expected}." self.feedback_color = (240, 75, 75) self.input_box.clear()

def handle_event(self, event): self.input_box.handle_event(event)

if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN: self.submit_answer()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1: if self.submit_btn.collidepoint(event.pos):
self.submit_answer()

def update(self): elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000 self.time_remaining = max(0,
self.time_limit - elapsed)

if self.time_remaining <= 0: self.total_attempts += 1 self.feedback_msg = "TIME UP! Try the next card."
self.feedback_color = (240, 180, 70) self.generate_new_card()

def render(self, screen): screen.fill((25, 29, 37))

title_surf = self.font_title.render( "Math Flashcards Arena", True, (245, 245, 245) ) screen.blit( title_surf, ( self.width //
2 - title_surf.get_width() // 2, 18 ) )

score_surf = self.font_hud.render( f"Score: {self.score} / {self.total_attempts}", True, (255, 220, 80) ) screen.blit(
score_surf, ( self.width // 2 - score_surf.get_width() // 2, 58 ) )

card_rect = pygame.Rect( self.width // 2 - 130, 95, 260, 110 )

pygame.draw.rect( screen, (240, 242, 245), card_rect, border_radius=12 )

pygame.draw.rect( screen, (85, 120, 175), card_rect, width=3, border_radius=12 )

card_str = f"{self.num_a} {self.operator} {self.num_b}" card_surf = self.font_card.render( card_str, True, (25, 30, 42)
)

screen.blit( card_surf, ( card_rect.centerx - card_surf.get_width() // 2, card_rect.centery - card_surf.get_height() // 2 )
)

# Timer bar timer_rect = pygame.Rect( self.width // 2 - 130, 212, 260, 10 )

pygame.draw.rect( screen, (65, 70, 80), timer_rect, border_radius=5 )

timer_width = int( timer_rect.width * (self.time_remaining / self.time_limit) )

if timer_width > 0: pygame.draw.rect( screen, (80, 200, 110), pygame.Rect( timer_rect.x, timer_rect.y, timer_width,
timer_rect.height ), border_radius=5 )

timer_text = self.font_btn.render( f"Time: {self.time_remaining:.1f}s", True, (220, 225, 230) )

screen.blit( timer_text, ( self.width // 2 - timer_text.get_width() // 2, 330 ) )

self.input_box.render(screen)

pygame.draw.rect( screen, (45, 140, 80), self.submit_btn, border_radius=6 )

pygame.draw.rect( screen, (215, 225, 220), self.submit_btn, width=2, border_radius=6 )

```

```

btn_txt = self.font_btn.render("SUBMIT", True, (255, 255, 255) )

screen.blit( btn_txt, ( self.submit_btn.centerx - btn_txt.get_width() // 2, self.submit_btn.centery - btn_txt.get_height() // 2 ) )

msg_surf = self.font_hud.render( self.feedback_msg, True, self.feedback_color )

screen.blit( msg_surf, ( self.width // 2 - msg_surf.get_width() // 2, 295 ) )

```

i thing you give full modified code

ASSISTANT

Yes I agree. Since you're a beginner, **full modified code is safer** and reduces the chance of indentation mistakes.

For **Task 3**, we'll keep everything you already have from Tasks 1 and 2 and add the streak system.

STEP 33 — Replace the complete `game_engine.py`

Press **Ctrl + A** in `game_engine.py`, delete everything, and paste this:

```

PYTHON

import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0

        # Streak system
        self.streak = 0
        self.streak_multiplier = 1

        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        # Timer system
        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator = random.choice(["+", "-", "*"])

```

```

        if self.operator == "-" and self.num_a < self.num_b:
            self.num_a, self.num_b = self.num_b, self.num_a

        self.input_box.clear()

        # Reset timer for new question
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

    def compute_expected_answer(self):
        if self.operator == "+":
            return self.num_a + self.num_b
        elif self.operator == "-":
            return self.num_a - self.num_b
        elif self.operator == "*":
            return self.num_a * self.num_b

    def submit_answer(self):
        val_str = self.input_box.text.strip()

        if not val_str or val_str == "-":
            self.feedback_msg = "Type an answer first!"
            self.feedback_color = (240, 175, 40)
            return

        user_answer = int(val_str)
        expected = self.compute_expected_answer()
        self.total_attempts += 1

        if user_answer == expected:
            # Increase consecutive correct streak
            self.streak += 1

            # Maximum multiplier is 3x
            self.streak_multiplier = min(self.streak, 3)

            # Add points according to multiplier
            self.score += self.streak_multiplier

            self.feedback_msg = (
                f"CORRECT! +{self.streak_multiplier} "
                f"({self.streak} streak) "
                f"{self.num_a} {self.operator} {self.num_b} = {expected}"
            )

            self.feedback_color = (80, 230, 110)
            self.generate_new_card()

        else:
            # Wrong answer resets the streak
            self.streak = 0
            self.streak_multiplier = 1

            self.feedback_msg = f"WRONG! Expected {expected}."
            self.feedback_color = (240, 75, 75)
            self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
            self.submit_answer()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_answer()

```

```

def update(self):
    elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000
    self.time_remaining = max(0, self.time_limit - elapsed)

    if self.time_remaining <= 0:
        self.total_attempts += 1

        # Timeout resets the streak
        self.streak = 0
        self.streak_multiplier = 1

        self.feedback_msg = "TIME UP! Streak reset."
        self.feedback_color = (240, 180, 70)

        self.generate_new_card()

def render(self, screen):
    screen.fill((25, 29, 37))

    title_surf = self.font_title.render(
        "Math Flashcards Arena",
        True,
        (245, 245, 245)
    )

    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            18
        )
    )

    score_surf = self.font_hud.render(
        f"Score: {self.score} / {self.total_attempts}",
        True,
        (255, 220, 80)
    )

    screen.blit(
        score_surf,
        (
            self.width // 2 - score_surf.get_width() // 2,
            58
        )
    )

    # Display current streak and multiplier
    streak_surf = self.font_hud.render(
        f"Streak: {self.streak} Multiplier: {self.streak_multiplier}x",
        True,
        (180, 220, 255)
    )

    screen.blit(
        streak_surf,
        (
            self.width // 2 - streak_surf.get_width() // 2,
            78
        )
    )

    card_rect = pygame.Rect(
        self.width // 2 - 130,
        105,

```

```

        260,
        110
    )

    pygame.draw.rect(
        screen,
        (240, 242, 245),
        card_rect,
        border_radius=12
    )

    pygame.draw.rect(
        screen,
        (85, 120, 175),
        card_rect,
        width=3,
        border_radius=12
    )

    card_str = f"{self.num_a} {self.operator} {self.num_b}"

    card_surf = self.font_card.render(
        card_str,
        True,
        (25, 30, 42)
    )

    screen.blit(
        card_surf,
        (
            card_rect.centerx - card_surf.get_width() // 2,
            card_rect.centery - card_surf.get_height() // 2
        )
    )

    # Timer bar
    timer_rect = pygame.Rect(
        self.width // 2 - 130,
        222,
        260,
        10
    )

    pygame.draw.rect(
        screen,
        (65, 70, 80),
        timer_rect,
        border_radius=5
    )

    timer_width = int(
        timer_rect.width *
        (self.time_remaining / self.time_limit)
    )

    if timer_width > 0:
        pygame.draw.rect(
            screen,
            (80, 200, 110),
            pygame.Rect(
                timer_rect.x,
                timer_rect.y,
                timer_width,
                timer_rect.height
            ),
            border_radius=5

```

```

    )

    timer_text = self.font_btn.render(
        f"Time: {self.time_remaining:.1f}s",
        True,
        (220, 225, 230)
    )

    screen.blit(
        timer_text,
        (
            self.width // 2 - timer_text.get_width() // 2,
            340
        )
    )

    self.input_box.render(screen)

    pygame.draw.rect(
        screen,
        (45, 140, 80),
        self.submit_btn,
        border_radius=6
    )

    pygame.draw.rect(
        screen,
        (215, 225, 220),
        self.submit_btn,
        width=2,
        border_radius=6
    )

    btn_txt = self.font_btn.render(
        "SUBMIT",
        True,
        (255, 255, 255)
    )

    screen.blit(
        btn_txt,
        (
            self.submit_btn.centerx - btn_txt.get_width() // 2,
            self.submit_btn.centery - btn_txt.get_height() // 2
        )
    )

    msg_surf = self.font_hud.render(
        self.feedback_msg,
        True,
        self.feedback_color
    )

    screen.blit(
        msg_surf,
        (
            self.width // 2 - msg_surf.get_width() // 2,
            295
        )
    )

```

What changed in Task 3?

The important new behavior is:

Correct answers:

TEXT

```
1 s t   c o r r e c t   ! '   + 1
2 n d   c o r r e c t   ! '   + 2
3 r d   c o r r e c t   ! '   + 3
4 t h   c o r r e c t   ! '   + 3
5 t h   c o r r e c t   ! '   + 3
```

Wrong answer:

TEXT

```
S t r e a k   ! '   0
M u l t i p l i e r   ! '   1 x
```

Timeout:

TEXT

```
S t r e a k   ! '   0
M u l t i p l i e r   ! '   1 x
```

I also moved the timer slightly so the **streak information and timer don't overlap**.

& Don't commit yet

After pasting:

1. Press **Ctrl + S**
2. **Don't run the game yet.**
3. Reply **DONE**.

Then we'll test Task 3 properly before committing it.

USER

done

ASSISTANT

Great ' Task 3 code is saved.

STEP 34 — Run the game

In the VS Code terminal, run:

BASH

```
python main.py
```

Now we need to test the **streak multiplier**.

Test 1 — Consecutive correct answers

For the first question:

1. Solve the math question.
2. Enter the correct answer.
3. Look at the top of the game.

You should see something like:

TEXT

Streak: 1 Multiplier: 1x

Then answer the **next question correctly**.

You should see:

TEXT

Streak: 2 Multiplier: 2x

Then answer another one correctly:

TEXT

Streak: 3 Multiplier: 3x

The score should increase by **1, then 2, then 3**.

Test 2 — Wrong answer

After getting a few correct answers, intentionally enter a **wrong answer**.

You should see:

TEXT

Streak: 0 Multiplier: 1x

Test 3 — Timeout

Then let one question reach zero.

The streak should reset to:

TEXT

Streak: 0 Multiplier: 1x

For now, test only the first part: get 3 consecutive correct answers.

Then tell me what you see for the streak/multiplier, or send me a screenshot.

ASSISTANT

Good ~~one~~ screenshot confirms **Task 3 timeout behavior is working**:

- Streak: 0
- Multiplier: 1x
- TIME UP! Streak reset.
- Timer restarted at 5.5s

So the timeout part is correct. '

STEP 35 — Test the streak multiplier

Now we need to test the **correct-answer streak**.

You currently have:

TEXT

3 * 7

and the answer is **21**.

1. Enter **21** if it isn't already there.
2. Click **SUBMIT** quickly.
3. The next question will appear.
4. Solve it correctly and submit quickly.
5. Do this for **3 correct answers in a row**.

You should eventually see:

TEXT

Streak: 3 Multiplier: 3x

And the score should increase:

TEXT

+1
+2
+3

& **Answer quickly**, because the timer is only 10 seconds.

Send me a screenshot once you reach **Streak: 3 / Multiplier: 3x**.

ASSISTANT

Excellent! **Task 3 is working correctly.**

Your screenshot clearly shows:

- **Score: 9 / 4**
- **Streak: 4**
- **Multiplier: 3x**
- **CORRECT! +3**
- The timer is still running.

That means the streak multiplier is working exactly as intended:

1st correct !' +1 !' 1x 2nd correct !' +2 !' 2x 3rd correct !' +3 !' 3x 4th correct !' +3 !' 3x '

So Task 3 has passed the main test.

STEP 36 — Close the game

Close the Pygame window using the **X** in the top-right corner.

Don't change the code anymore.

After closing it, tell me **DONE**. Then we'll commit Task 3 separately.

USER

done

ASSISTANT

Perfect ' **Task 3 testing is complete.**

STEP 37 — Commit Task 3

In the VS Code terminal, type:

BASH

```
git status
```

Press **Enter**.

We expect:

TEXT

```
Changes not staged for commit:
  modified:   game/game_engine.py
```

game/ __pycache__ / may also appear. **Do not add it.**

Send me the `git status` output first. Then I'll give you the exact commit command.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is ahead of
'origin/main' by 2 commits. (use "git push" to publish your local commits)
```

```
Changes not staged for commit: (use "git add <file>..." to update what will be committed)
(use "git restore <file>..." to discard changes in working directory)
    modified:   game/game_engine.py
```

```
Untracked files: (use "git add <file>..." to include in what will be committed)
    game/__pycache__/
```

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect ~~0=0~~ Our `git status` is correct.

STEP 38 — Stage Task 3

Type:

BASH

```
git add game/game_engine.py
```

Press **Enter**.

Then type:

BASH

```
git status
```

You should see:

TEXT

```
Changes to be committed:
  modified:   game/game_engine.py
```

game/___pycache___/ should remain **untracked** and must not be committed.

Don't commit yet. Send me the new `git status` output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git add game/game_engine.py
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is ahead of 'origin/main' by 2 commits. (use "git push" to publish your local commits)

Changes to be committed: (use "git restore --staged <file>..." to unstage)
  modified:   game/game_engine.py

Untracked files: (use "git add <file>..." to include in what will be committed)
  game/pycache/

PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect ' Task 3 is staged correctly.

STEP 39 — Commit Task 3

Now type exactly:

BASH

```
git commit -m "Add streak score multipliers"
```

Press **Enter**.

You should see something like:

TEXT

```
[main XXXXXXX] Add streak score multipliers
1 file changed, ...
```

& **Do not run `git push` yet.**

After the commit finishes, send me the output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git commit -m "Add streak score multipliers"
[main f77e875] Add streak score multipliers
1 file changed, 53 insertions(+), 7 deletions(-)
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Excellent! **Task 3 is now committed successfully.**

Your local Git history now has **3 separate commits**:

1. `Fix arithmetic calculation bug`
2. `Add per-question timer`
3. `Add streak score multipliers`

And we still have **not pushed** anything to GitHub, which is correct for now.

Next: Task 4 — Division

The README requires adding **division with clean integer quotients**, meaning questions such as:

TEXT

$12 \div 3 = 4$

but never:

TEXT

$12 \div 5 = 2.4$

We need to modify the question generation so every division answer is a whole number.

STEP 40 — Start Task 4

Before changing anything, we will use your current full `game_engine.py`.

You already know the process, so this time we can use the **full-code method** you prefer.

Don't edit anything yet.

Reply **READY** and I'll give you the complete Task 4 version, keeping Tasks 1, 2, and 3 intact.

USER

```
import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0

        # Streak system
        self.streak = 0
        self.streak_multiplier = 1

        self.feedback_msg = "Solve the card and press Enter!"
        self.feedback_color = (200, 205, 215)

        # Timer system
        self.time_limit = 10
        self.time_remaining = self.time_limit
        self.timer_start = pygame.time.get_ticks()

        self.num_a = 0
        self.num_b = 0
        self.operator = "+"

        box_w, box_h = 130, 44
        self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
        self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

        self.font_title = pygame.font.SysFont(None, 38)
        self.font_hud = pygame.font.SysFont(None, 26)
        self.font_card = pygame.font.SysFont(None, 56)
        self.font_btn = pygame.font.SysFont(None, 24)

        self.generate_new_card()

    def generate_new_card(self):
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)
        self.operator =
```

```

random.choice(["+", "-", "*"])

if self.operator == "-" and self.num_a < self.num_b: self.num_a, self.num_b = self.num_b, self.num_a

self.input_box.clear()

# Reset timer for new question self.time_remaining = self.time_limit self.timer_start = pygame.time.get_ticks()

def compute_expected_answer(self): if self.operator == "+": return self.num_a + self.num_b elif self.operator == "-":
return self.num_a - self.num_b elif self.operator == "*": return self.num_a * self.num_b

def submit_answer(self): val_str = self.input_box.text.strip()

if not val_str or val_str == "-": self.feedback_msg = "Type an answer first!" self.feedback_color = (240, 175, 40) return

user_answer = int(val_str) expected = self.compute_expected_answer() self.total_attempts += 1

if user_answer == expected: # Increase consecutive correct streak self.streak += 1

# Maximum multiplier is 3x self.streak_multiplier = min(self.streak, 3)

# Add points according to multiplier self.score += self.streak_multiplier

self.feedback_msg = ( f"CORRECT! +{self.streak_multiplier} " f"({self.streak} streak) " f"{self.num_a} {self.operator}
{self.num_b} = {expected}" )

self.feedback_color = (80, 230, 110) self.generate_new_card()

else: # Wrong answer resets the streak self.streak = 0 self.streak_multiplier = 1

self.feedback_msg = f"WRONG! Expected {expected}." self.feedback_color = (240, 75, 75) self.input_box.clear()

def handle_event(self, event): self.input_box.handle_event(event)

if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN: self.submit_answer()

elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1: if self.submit_btn.collidepoint(event.pos):
self.submit_answer()

def update(self): elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000 self.time_remaining = max(0,
self.time_limit - elapsed)

if self.time_remaining <= 0: self.total_attempts += 1

# Timeout resets the streak self.streak = 0 self.streak_multiplier = 1

self.feedback_msg = "TIME UP! Streak reset." self.feedback_color = (240, 180, 70)

self.generate_new_card()

def render(self, screen): screen.fill((25, 29, 37))

title_surf = self.font_title.render( "Math Flashcards Arena", True, (245, 245, 245) )

screen.blit( title_surf, ( self.width // 2 - title_surf.get_width() // 2, 18 ) )

score_surf = self.font_hud.render( f"Score: {self.score} / {self.total_attempts}", True, (255, 220, 80) )

screen.blit( score_surf, ( self.width // 2 - score_surf.get_width() // 2, 58 ) )

# Display current streak and multiplier streak_surf = self.font_hud.render( f"Streak: {self.streak} Multiplier:
{self.streak_multiplier}x", True, (180, 220, 255) )

screen.blit( streak_surf, ( self.width // 2 - streak_surf.get_width() // 2, 78 ) )

card_rect = pygame.Rect( self.width // 2 - 130, 105, 260, 110 )

pygame.draw.rect( screen, (240, 242, 245), card_rect, border_radius=12 )

```

```

pygame.draw.rect( screen, (85, 120, 175), card_rect, width=3, border_radius=12 )

card_str = f"{self.num_a} {self.operator} {self.num_b}"

card_surf = self.font_card.render( card_str, True, (25, 30, 42) )

screen.blit( card_surf, ( card_rect.centerx - card_surf.get_width() // 2, card_rect.centery - card_surf.get_height() // 2 )
)

# Timer bar timer_rect = pygame.Rect( self.width // 2 - 130, 222, 260, 10 )

pygame.draw.rect( screen, (65, 70, 80), timer_rect, border_radius=5 )

timer_width = int( timer_rect.width * (self.time_remaining / self.time_limit) )

if timer_width > 0: pygame.draw.rect( screen, (80, 200, 110), pygame.Rect( timer_rect.x, timer_rect.y, timer_width,
timer_rect.height ), border_radius=5 )

timer_text = self.font_btn.render( f"Time: {self.time_remaining:.1f}s", True, (220, 225, 230) )

screen.blit( timer_text, ( self.width // 2 - timer_text.get_width() // 2, 340 ) )

self.input_box.render(screen)

pygame.draw.rect( screen, (45, 140, 80), self.submit_btn, border_radius=6 )

pygame.draw.rect( screen, (215, 225, 220), self.submit_btn, width=2, border_radius=6 )

btn_txt = self.font_btn.render( "SUBMIT", True, (255, 255, 255) )

screen.blit( btn_txt, ( self.submit_btn.centerx - btn_txt.get_width() // 2, self.submit_btn.centery - btn_txt.get_height() //
2 ) )

msg_surf = self.font_hud.render( self.feedback_msg, True, self.feedback_color )

screen.blit( msg_surf, ( self.width // 2 - msg_surf.get_width() // 2, 295 ) )

```

modify task 4 and give full code

ASSISTANT

Yes, I will modify **only Task 4** while keeping Tasks 1, 2, and 3 exactly as they are.

For Task 4, division will be generated so the answer is always a **whole integer**. For example, the game can generate $12 \div 3$, but not $12 \div 5$.

STEP 40 — Replace `game_engine.py`

In VS Code:

Ctrl + A !' Delete !' paste this complete code !' Ctrl + S

PYTHON

```

import random
import pygame
from game.text_box import TextBox

class GameEngine:
    def __init__(self, width, height):
        self.width = width
        self.height = height

        self.score = 0
        self.total_attempts = 0

```

```

# Streak system
self.streak = 0
self.streak_multiplier = 1

self.feedback_msg = "Solve the card and press Enter!"
self.feedback_color = (200, 205, 215)

# Timer system
self.time_limit = 10
self.time_remaining = self.time_limit
self.timer_start = pygame.time.get_ticks()

self.num_a = 0
self.num_b = 0
self.operator = "+"

box_w, box_h = 130, 44
self.input_box = TextBox(width // 2 - 110, 230, box_w, box_h)
self.submit_btn = pygame.Rect(width // 2 + 30, 230, 90, box_h)

self.font_title = pygame.font.SysFont(None, 38)
self.font_hud = pygame.font.SysFont(None, 26)
self.font_card = pygame.font.SysFont(None, 56)
self.font_btn = pygame.font.SysFont(None, 24)

self.generate_new_card()

def generate_new_card(self):
    self.operator = random.choice(["+", "-", "*", "/"])

    if self.operator == "/":
        # Generate a clean integer division problem.
        # Example: 4 * 3 = 12, so the question becomes 12 / 3 = 4.
        divisor = random.randint(2, 12)
        quotient = random.randint(2, 10)

        self.num_b = divisor
        self.num_a = divisor * quotient

    else:
        self.num_a = random.randint(3, 15)
        self.num_b = random.randint(2, 12)

        if self.operator == "-" and self.num_a < self.num_b:
            self.num_a, self.num_b = self.num_b, self.num_a

    self.input_box.clear()

    # Reset timer for new question
    self.time_remaining = self.time_limit
    self.timer_start = pygame.time.get_ticks()

def compute_expected_answer(self):
    if self.operator == "+":
        return self.num_a + self.num_b
    elif self.operator == "-":
        return self.num_a - self.num_b
    elif self.operator == "*":
        return self.num_a * self.num_b
    elif self.operator == "/":
        return self.num_a // self.num_b

def submit_answer(self):
    val_str = self.input_box.text.strip()

    if not val_str or val_str == "-":

```

```

        self.feedback_msg = "Type an answer first!"
        self.feedback_color = (240, 175, 40)
        return

    user_answer = int(val_str)
    expected = self.compute_expected_answer()
    self.total_attempts += 1

    if user_answer == expected:
        # Increase consecutive correct streak
        self.streak += 1

        # Maximum multiplier is 3x
        self.streak_multiplier = min(self.streak, 3)

        # Add points according to multiplier
        self.score += self.streak_multiplier

        self.feedback_msg = (
            f"CORRECT! +{self.streak_multiplier} "
            f"({self.streak} streak) "
            f"{self.num_a} {self.operator} {self.num_b} = {expected}"
        )

        self.feedback_color = (80, 230, 110)
        self.generate_new_card()

    else:
        # Wrong answer resets the streak
        self.streak = 0
        self.streak_multiplier = 1

        self.feedback_msg = f"WRONG! Expected {expected}."
        self.feedback_color = (240, 75, 75)
        self.input_box.clear()

    def handle_event(self, event):
        self.input_box.handle_event(event)

        if event.type == pygame.KEYDOWN and event.key == pygame.K_RETURN:
            self.submit_answer()

        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            if self.submit_btn.collidepoint(event.pos):
                self.submit_answer()

    def update(self):
        elapsed = (pygame.time.get_ticks() - self.timer_start) / 1000
        self.time_remaining = max(0, self.time_limit - elapsed)

        if self.time_remaining <= 0:
            self.total_attempts += 1

            # Timeout resets the streak
            self.streak = 0
            self.streak_multiplier = 1

            self.feedback_msg = "TIME UP! Streak reset."
            self.feedback_color = (240, 180, 70)

            self.generate_new_card()

    def render(self, screen):
        screen.fill((25, 29, 37))

        title_surf = self.font_title.render(

```

```

        "Math Flashcards Arena",
        True,
        (245, 245, 245)
    )

    screen.blit(
        title_surf,
        (
            self.width // 2 - title_surf.get_width() // 2,
            18
        )
    )

    score_surf = self.font_hud.render(
        f"Score: {self.score} / {self.total_attempts}",
        True,
        (255, 220, 80)
    )

    screen.blit(
        score_surf,
        (
            self.width // 2 - score_surf.get_width() // 2,
            58
        )
    )

    # Display current streak and multiplier
    streak_surf = self.font_hud.render(
        f"Streak: {self.streak} Multiplier: {self.streak_multiplier}x",
        True,
        (180, 220, 255)
    )

    screen.blit(
        streak_surf,
        (
            self.width // 2 - streak_surf.get_width() // 2,
            78
        )
    )

    card_rect = pygame.Rect(
        self.width // 2 - 130,
        105,
        260,
        110
    )

    pygame.draw.rect(
        screen,
        (240, 242, 245),
        card_rect,
        border_radius=12
    )

    pygame.draw.rect(
        screen,
        (85, 120, 175),
        card_rect,
        width=3,
        border_radius=12
    )

    card_str = f"{self.num_a} {self.operator} {self.num_b}"

```

```

card_surf = self.font_card.render(
    card_str,
    True,
    (25, 30, 42)
)

screen.blit(
    card_surf,
    (
        card_rect.centerx - card_surf.get_width() // 2,
        card_rect.centery - card_surf.get_height() // 2
    )
)

# Timer bar
timer_rect = pygame.Rect(
    self.width // 2 - 130,
    222,
    260,
    10
)

pygame.draw.rect(
    screen,
    (65, 70, 80),
    timer_rect,
    border_radius=5
)

timer_width = int(
    timer_rect.width *
    (self.time_remaining / self.time_limit)
)

if timer_width > 0:
    pygame.draw.rect(
        screen,
        (80, 200, 110),
        pygame.Rect(
            timer_rect.x,
            timer_rect.y,
            timer_width,
            timer_rect.height
        ),
        border_radius=5
    )

timer_text = self.font_btn.render(
    f"Time: {self.time_remaining:.1f}s",
    True,
    (220, 225, 230)
)

screen.blit(
    timer_text,
    (
        self.width // 2 - timer_text.get_width() // 2,
        340
    )
)

self.input_box.render(screen)

pygame.draw.rect(
    screen,
    (45, 140, 80),

```

```

        self.submit_btn,
        border_radius=6
    )

    pygame.draw.rect(
        screen,
        (215, 225, 220),
        self.submit_btn,
        width=2,
        border_radius=6
    )

    btn_txt = self.font_btn.render(
        "SUBMIT",
        True,
        (255, 255, 255)
    )

    screen.blit(
        btn_txt,
        (
            self.submit_btn.centerx - btn_txt.get_width() // 2,
            self.submit_btn.centery - btn_txt.get_height() // 2
        )
    )

    msg_surf = self.font_hud.render(
        self.feedback_msg,
        True,
        self.feedback_color
    )

    screen.blit(
        msg_surf,
        (
            self.width // 2 - msg_surf.get_width() // 2,
            295
        )
    )
)

```

What Task 4 changed

The important part is:

PYTHON

```
self.operator = random.choice(["+", "-", "*", "/"])
```

For division, we generate:

PYTHON

```
divisor = random.randint(2, 12)
quotient = random.randint(2, 10)

self.num_b = divisor
self.num_a = divisor * quotient
```

So if:

TEXT

```
divisor = 4
quotient = 6
```

the game creates:

TEXT

24 / 4 = 6

Therefore, **division always has a clean integer answer.**

Don't run it yet

After you paste the code:

Ctrl + S

Then reply:

DONE

We'll test Task 4 before making the fourth Git commit.

USER

total how many task is there

ASSISTANT

There are **4 tasks/features** in the README for this lab.

1. ' Task 1 — Fix arithmetic calculation bug

- +, -, * calculate correctly
- Committed: Fix arithmetic calculation bug

2. ' Task 2 — Per-question timer

- 10-second countdown
- Timer bar
- Timeout counts as an attempt
- Committed: Add per-question timer

3. ' Task 3 — Consecutive correct streak multiplier

- 1x! 2x! 3x
- Wrong answer/timeout resets streak
- Committed: Add streak score multipliers

4. Ø=Task 4 — Division with integer answers

- Add /
- Ensure every division has a whole-number answer
- **Currently working on this one**

So after **Task 4**, the coding tasks are finished. Ø<β

Then we'll do the remaining lab requirements:

- test the final game
- record the **after-changes 10-second video**
- check Git commits
- push your code to your personal GitHub repo

- prepare the chat history/submission materials.

USER

task 4 paste done

ASSISTANT

Perfect! Task 4 code is pasted and saved.

STEP 41 — Test Task 4

In the VS Code terminal, run:

BASH

```
python main.py
```

Now look at the math cards.

We need to find a **division (/)** question.

For example:

TEXT

12 / 3

or

TEXT

24 / 4

or

TEXT

30 / 5

The answer must always be a **whole number**.

Do this now

1. Start the game.
2. Wait until you get a question containing `/`.
3. **Don't answer it yet.**
4. Check that the division has no remainder.
5. Send me a screenshot showing the `/` question.

Don't commit Task 4 yet.

ASSISTANT

Excellent! Task 4 is working.

Your screenshot shows:

TEXT

8 / 2

Answer: 4

So this is a clean integer division. '

It also shows that the previous Task 3 functionality is still working:

- Streak: 5
- Multiplier: 3x
- CORRECT! +3

So **Tasks 1, 2, 3, and 4 are all working together.** Ø<ß

STEP 42 — Test the division answer

Now enter the correct answer for:

TEXT

8 / 2

The answer is:

TEXT

4

You already have 4 in the input box, so simply click **SUBMIT**.

You should get a **CORRECT** message and then a new card.

After that, **close the game**.

Tell me **DONE** when you've finished. Then we'll make the final Task 4 Git commit.

USER

done

ASSISTANT

Perfect ' **Task 4 testing is complete.**

Now let's make the required separate Git commit.

STEP 43 — Check Git status

In the VS Code terminal, type:

BASH

git status

Press **Enter**.

You should see:

TEXT

modified: game/game_engine.py

and possibly:

TEXT

game/__pycache__/
game/

Again, **do not add** `__pycache__`.

Send me the output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status
On branch main
Your branch is ahead of 'origin/main' by 3 commits. (use "git push" to publish your local commits)
```

```
Changes not staged for commit: (use "git add <file>..." to update what will be committed) (use "git restore <file>..." to discard changes in working directory)
modified:   game/game_engine.py
```

```
Untracked files: (use "git add <file>..." to include in what will be committed)
game/
```

```
no changes added to commit (use "git add" and/or "git commit -a")
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect! Task 4 is ready to commit.

STEP 44 — Stage Task 4

Type:

BASH

```
git add game/game_engine.py
```

Press **Enter**.

Then:

BASH

```
git status
```

You should see:

TEXT

```
Changes to be committed:
  modified:   game/game_engine.py
```

`game/` should remain untracked.

Don't commit yet. Send me the output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git add game/game_engine.py PS
C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git status On branch main Your branch is ahead of 'origin/main'
by 3 commits. (use "git push" to publish your local commits)

Changes to be committed: (use "git restore --staged <file>..." to unstage) modified:   game/game_engine.py

Untracked files: (use "git add <file>..." to include in what will be committed) game/pycache/

PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect ' Task 4 is staged correctly.

STEP 45 — Commit Task 4

Now type exactly:

BASH

```
git commit -m "Add integer division questions"
```

Press **Enter**.

You should see something similar to:

TEXT

```
[main XXXXXXX] Add integer division questions
1 file changed, ...
```

& **Do not run `git push` yet.**

Send me the commit output after it finishes.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git commit -m "Add integer division questions" [main
befc000] Add integer division questions 1 file changed, 18 insertions(+), 5 deletions(-) PS
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Excellent  **Task 4 is successfully committed.**

Your four required tasks are now complete:

Task	Feature	Commit
1	Fix arithmetic calculation	' Fix arithmetic calculation bug
2	Per-question timer	' Add per-question timer
3	Streak score multipliers	' Add streak score multipliers

Task	Feature	Commit
4	Integer division questions	' Add integer division questions

Your branch should now be **4 commits ahead of `origin/main`**.

What remains

We are **not finished with the lab submission yet**. The coding portion is complete, but we still need to do the submission requirements:

1. ' Before-change 10-second video — already recorded
2. ' Tasks 1–4 implemented
3. ~~Ø=~~Record **after-change 10-second video**
4. ~~Ø=~~Push your code to your personal GitHub repo
5. ~~Ø=~~Make sure the four commits are present
6. ~~Ø=~~Prepare the required chat/LLM history for submission
7. ~~Ø=~~Organize the Lab-4 submission folder

The handout specifically requires the before/after videos, updated code, and complete chat history.

Do not push yet.

STEP 46 — Check the final Git history

In the terminal, type:

```
BASH
git log --oneline --max-count=5
```

Send me the output.

I'll verify that all **4 task commits** are there before we move to the final video and GitHub push.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git log --oneline --max-count=5 bef000 (HEAD -> main)
Add integer division questions f77e875 Add streak score multipliers 34957d4 Add per-question timer 65a6f59 Fix
arithmetic calculation bug cdb1f0b (origin/main, origin/HEAD) Update game_engine.py PS
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

ASSISTANT

Perfect ' Your Git history is **exactly correct**.

You have all 4 task commits:

```
TEXT
bef000 Add integer division questions
f77e875 Add streak score multipliers
34957d4 Add per-question timer
65a6f59 Fix arithmetic calculation bug
```

And this line:

TEXT

```
cdb1f0b (origin/main, origin/HEAD) Update game_engine.py
```

shows that your original repository is still unchanged remotely. Your 4 changes are currently **only on your local `main` branch**, which is what we want before the final push.

STEP 47 — Final game test

Before pushing to GitHub, let's make sure the **final version** works.

Run:

BASH

```
python main.py
```

Test these 4 things quickly:

1. **Arithmetic** !' `+`, `-`, `*` calculate correctly.
2. **Timer** !' countdown bar works and timeout increases attempts.
3. **Streak** !' consecutive correct answers increase multiplier up to `3x`.
4. **Division** !' `/` questions have whole-number answers.

Important

This is also the perfect time to record your **after-changes 10-second video**, because the lab requires a second video showing the fixed/new features.

Don't push to GitHub yet.

First run the final game and tell me **DONE** when you're satisfied it works. Then I'll guide you through recording the final video.

USER

i want to take screen record

ASSISTANT

Yes ~~0=0m~~ You need to record the **after-changes video** now. The lab asks for a **10-second video after all changes**.

STEP 47 — Start the final game

In VS Code terminal:

BASH

```
python main.py
```

Wait until the game window opens.

STEP 48 — Start Windows screen recording

Since you're on Windows 11:

1. Press **`Windows + Shift + R`**

2. Windows will start screen recording.
3. Record the **Pygame game window** for about **10–15 seconds**.

During the recording, try to show these features:

- #1 Timer countdown/bar
- '—A division question such as 8 / 2
- Ø=3 Streak/multiplier such as 3x
- ' A correct answer

You don't need to demonstrate all four tasks perfectly in 10 seconds. The goal is to clearly show the **final modified game**.

After recording

Press **Windows + Shift + R** again to stop recording.

Then tell me **"recorded"**.

I'll guide you to find the recorded video and make sure it is suitable for submission.

USER

cut and edit this into 10 sec

ASSISTANT

Done. I trimmed your recording to **10 seconds** and kept the gameplay section showing the updated features.

Download the edited 10-second video

Use this as your **"10-sec video after changes"** for Lab 4.

USER

done what is the next thing we want to doo

ASSISTANT

Great Ø=3 **10-sec video after-change 10-second video is done**.

Now we need to do the next important part: **push your completed code to your personal GitHub repository**. The lab specifically says the code must be pushed to your **personal repo**, not the main `SETAPESU26` repo.

STEP 49 — Check where your code currently points

In the **VS Code terminal**, run:

BASH

```
git remote -v
```

You should see something like:

TEXT

```
origin https://github.com/SETAPESU26/61_math_flashcards.git
origin https://github.com/SETAPESU26/61_math_flashcards.git
```

Don't push anything yet.

Send me a screenshot of the output (or copy-paste it here), and I'll tell you the next step.

USER

```
PS C:\Users\Admin\Desktop\Lab4\61_math_flashcards> git remote -v origin
https://github.com/SETAPESU26/61_math_flashcards.git(fetch) origin
https://github.com/SETAPESU26/61_math_flashcards.git(push)
```

ASSISTANT

Perfect. ~~0=0ms~~ confirms an important thing:

Your `origin` is currently pointing to the **teacher/main repository**:

TEXT

```
https://github.com/SETAPESU26/61_math_flashcards.git
```

& **Do NOT run `git push` yet.** The lab says your completed code must be pushed to **your personal GitHub repository**, not the `SETAPESU26` main repository.

STEP 50 — Create your personal copy on GitHub

Open this repository in your browser:

On the GitHub page:

1. Look for the **Fork** button near the top-right.
2. Click **Fork**.
3. GitHub will show **Create a new fork**.
4. Under **Owner**, select **your own GitHub account**.
5. Keep the repository name as:

```
61_math_flashcards
```

6. Click **Create fork**.

After this, you should have a repository under **your own GitHub username**, for example:

TEXT

```
https://github.com/YOUR_USERNAME/61_math_flashcards
```

Stop there. Don't change anything in VS Code yet.

Tell me **“fork done”** after you see your personal repository.

USER

i already have a repository of this subject i already push the lab 1, 2, 3 lab in that my mam told to attach lab 4 for that

only

ASSISTANT

Ah, yes! ~~Oh~~ In that case, **do not create a new repository or fork anything.**

You already have your **personal college repository** where Lab 1, Lab 2, and Lab 3 are stored. We should add **Lab 4 to that same repository**, exactly as your mam instructed.

The only thing we need now is to connect this Lab 4 project to your existing personal repository.

STEP 50 — Tell me your existing repository URL

Open your GitHub repository where you already uploaded **Lab 1, Lab 2, and Lab 3** and copy its URL.

It will look something like:

TEXT

```
https://github.com/YourUsername/YourRepositoryName
```

Send me that URL here.

Don't run any Git commands yet. Once you send it, I'll give you exactly the next command to connect this Lab 4 folder to that repository.

USER

```
https://github.com/PurvikSH/SE_PES1UG25CS838
```

ASSISTANT

Perfect ~~Oh~~ I've checked your repository. It already contains your **Lab3** and other SE files, so we will add **Lab4 to this same repository.**

We should **not change the remote of your current `61\_math\_flashcards` folder** and push it directly, because that could put the files in the wrong place.

STEP 51 — Clone your personal SE repository

We'll first make a local copy of your existing repository.

In your **VS Code terminal**, you are currently here:

TEXT

```
C:\Users\Admin\Desktop\Lab4\61_math_flashcards>
```

Run:

BASH

```
cd ..
```

Then run:

BASH

```
git clone https://github.com/PurvikSH/SE_PES1UG25CS838.git
```

You should see something like:

TEXT

```
Cloning into 'SE_PES1UG25CS838'...
```

Then run:

BASH

```
dir
```

You should see both:

TEXT

```
61_math_flashcards
SE_PES1UG25CS838
```

& Stop here

Don't copy anything and don't commit yet.

After you run these commands, tell me **"done"** and I'll give you the next step to put the Lab 4 files into the correct Lab-4 folder.

USER

```
PS C:\Users\Admin\Desktop\Lab4> dir
```

```
Directory: C:\Users\Admin\Desktop\Lab4
```

Mode	LastWriteTime	Length	Name						
PM	10/7/2026 5:22 PM		61_math_flashcards	d-----					
			SE_PES1UG25CS838						

ASSISTANT

Perfect! That is exactly what we wanted.

You now have:

TEXT

```
Lab4
% % % 6 1 _ m a t h _ f l a s h c a r d s
% % % S E _ P E S 1 U G 2 5 C S 8 3 8
```

STEP 52 — Check your existing Lab folders

Now go inside your personal repository:

BASH

```
cd SE_PES1UG25CS838
```

Then:

```
BASH
dir
```

Don't copy anything yet.

Send me the output of `dir`. Then I'll tell you exactly where to create the **Lab-4** folder so we don't disturb your Lab 1, 2, and 3 work.

USER

```
PS C:\Users\Admin\Desktop\Lab4> cd SE_PES1UG25CS838 PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> dir
```

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838

Mode	LastWriteTime	Length	Name	----	-----	d----	10/7/2026 5:22
PM	Lab3 -a----	10/7/2026 5:22 PM	852673	Lab_2_jira_PES1UG25CS838.pdf	-a----		
10/7/2026 5:22 PM	194947	SE_Requirements_Table.pdf	-a----	10/7/2026 5:22 PM		48138	
SE_UML_diagram.pdf	-a----	10/7/2026 5:22 PM	215277	SE_UseCase_Flow.pdf			

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Perfect ~~0=0~~ Your personal repository currently has:

TEXT

```
SE_PES1UG25CS838
% % %   L a b 3
% % %   L a b _ 2 _ j i r a _ P E S 1 U G 2 5 C S 8 3 8 . p d f
% % %   S E _ R e q u i r e m e n t s _ T a b l e . p d f
% % %   S E _ U M L _ d i a g r a m . p d f
% % %   S E _ U s e C a s e _ F l o w . p d f
```

So we can safely add a new folder:

TEXT

```
Lab-4
```

STEP 53 — Create the Lab-4 folder

You are already inside:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

Run:

BASH

```
mkdir Lab-4
```

Then check:

```
BASH
dir
```

You should see:

```
TEXT
Lab3
Lab-4
Lab_2_jira_PES1UG25CS838.pdf
...
```

Don't put any files into it yet.

After you run it, tell me **"done"**. Then I'll show you how to copy your completed Math Flashcards project into `Lab-4` without accidentally copying unnecessary files like `.git` or `__pycache__`.

```
USER
S C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>mkdir Lab-4

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838

Mode                LastWriteTime         Length Name ----
PM                Lab-4
10/7/2026  5:24 PM

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>dir

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838

Mode                LastWriteTime         Length Name ----
PM                Lab-4 d----- 10/7/2026  5:22 PM
Lab_2_jira_PES1UG25CS838.pdf-a--- 10/7/2026  5:22 PM    194947 SE_Requirements_Table.pdf-a---
10/7/2026  5:22 PM    48138 SE_UML_diagram.pdf-a--- 10/7/2026  5:22 PM    215277
SE_UseCase_Flow.pdf
```

ASSISTANT

Perfect ' The `Lab-4` folder has been created correctly.

STEP 54 — Copy the completed project into Lab-4

We want the structure to look like this:

```
TEXT
SE_PES1UG25CS838
% % %   L a b - 4
% % %   6 1 _ m a t h _ f l a s h c a r d s
% % %   g a m e
% % %   m a i n . p y
% % %   R E A D M E . m d
```

We **do not** want to copy the `.git` folder from the original project, because your personal repository already has its own Git history.

From your current location:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

run:

POWERSHELL

```
Copy-Item ..\61_math_flashcards .\Lab-4\61_math_flashcards -Recurse -Exclude ".git"
```

Then run:

POWERSHELL

```
dir .\Lab-4\61_math_flashcards
```

You should see files/folders such as:

TEXT

```
game
main.py
README.md
```

Don't commit or push yet.

After you run these two commands, tell me **"done"**.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> Copy-Item ..\61_math_flashcards
.\Lab-4\61_math_flashcards -Recurse -Exclude ".git" PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> dir
.\Lab-4\61_math_flashcards

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4\61_math_flashcards

Mode                LastWriteTime         Length Name ----
-----
PM                .git d-----      10/7/2026   5:44 PM          game -a----      10/7/2026   3:21 PM          705 main.py
-a----      10/7/2026   3:21 PM          3702 README.md

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

I see one small problem & pthe `.git` folder was copied inside `Lab-4\61_math_flashcards`.

We **must remove that `.git` folder**, because your main repository `SE_PES1UG25CS838` should have only **one Git repository**.

STEP 55 — Remove the extra `.git` folder

You are currently here:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

Run:

POWERSHELL

```
Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse -Force
```

Then check:

POWERSHELL

```
dir .\Lab-4\61_math_flashcards -Force
```

You should see:

TEXT

```
game
main.py
README.md
```

and **you should NOT see** `.git`.

Don't do anything else yet. Tell me **"done"** after this.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse -Force
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> game
```

> main.py > README.md

game : The term 'game' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again. At line:1 char:1

- game
- ~~~~
 - CategoryInfo : ObjectNotFound: (game:String) [], CommandNotFoundException
 - FullyQualifiedErrorId : CommandNotFoundException

main.py : The term 'main.py' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again. At line:2 char:1

- main.py
- ~~~~
 - CategoryInfo : ObjectNotFound: (main.py:String) [], CommandNotFoundException
 - FullyQualifiedErrorId : CommandNotFoundException

README.md : The term 'README.md' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again. At line:3 char:1

- README.md
- ~~~~~~
 - CategoryInfo : ObjectNotFound: (README.md:String) [], CommandNotFoundException
 - FullyQualifiedErrorId : CommandNotFoundException

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> ^C PS
```

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse
```

-Force Remove-Item : Cannot find path

'C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4\61_math_flashcards\.git'because it does not exist. At line:1 char:1

- Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse -Force
- ~~~~~
 - CategoryInfo : ObjectNotFound: (C:\Users\Admin\...\flashcards\.git:String) [Remove-Item], ItemNotFound

Exception

- FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.RemoveItemCommand

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse

-Force Remove-Item : Cannot find path

'C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4\61_math_flashcards\.git'because it does not exist. At line:1 char:1

- Remove-Item .\Lab-4\61_math_flashcards\.git -Recurse -Force
- ~~~~~
 - CategoryInfo : ObjectNotFound: (C:\Users\Admin\...\flashcards\.git:String) [Remove-Item], ItemNotFound

Exception

- FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.RemoveItemCommand

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>

ASSISTANT

No problem ~~O=VM~~ did it correctly.

The **first** `Remove-Item` command succeeded. That's why the later commands say:

TEXT

Cannot find path ...\.git because it does not exist.

That means the `.git` folder is **already deleted**.

The `game`, `main.py`, and `README.md` errors happened because you typed those filenames directly into PowerShell. They were supposed to be things we *look at*, not commands. Nothing is damaged.

STEP 56 — Verify the folder

Run **only this command**:

POWERSHELL

```
dir .\Lab-4\61_math_flashcards -Force
```

You should see:

TEXT

```
game
main.py
README.md
```

and **no** `.git` folder.

Send me the output. Then we'll continue with the Git commit for Lab 4.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> dir .\Lab-4\61_math_flashcards -Force

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4\61_math_flashcards

Mode                LastWriteTime         Length Name ----
PM                10/7/2026  3:21 PM            705 main.py -a---
10/7/2026  3:21 PM            3702
README.md
```

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>

ASSISTANT

Perfect ' Everything is correct now.

Your Lab 4 project contains exactly:

TEXT

```
Lab-4
% % %   6 1 _ m a t h _ f l a s h c a r d s
% % %   g a m e
% % %   m a i n . p y
% % %   R E A D M E . m d
```

STEP 57 — Check Git status

Now we need to see what your personal repository detects.

Run:

POWERSHELL

```
git status
```

Don't run `git add` yet.

Send me the output, and I'll tell you exactly what to add and commit.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is up to date with
'origin/main'.

Untracked files: (use "git add <file>..." to include in what will be committed) Lab-4/

nothing added to commit but untracked files present (use "git add" to track) PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Perfect ' This is exactly what we expected.

Git sees the entire:

TEXT

Lab-4/

as a new folder. That means your existing Lab 1–3 files are untouched.

STEP 58 — Add Lab-4 to Git

Run **only**:

POWERSHELL

```
git add Lab-4
```

Then check what will be committed:

POWERSHELL

```
git status
```

You should see something similar to:

TEXT

```
Changes to be committed:
  new file:   Lab-4/61_math_flashcards/...
```

Don't commit yet.

Send me the `git status` output, and I'll check it before we create the Lab 4 commit.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git add Lab-4 PS
```

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is up to date with 'origin/main'.
```

```
Changes to be committed: (use "git restore --staged <file>..." to unstage) new file:
```

```
Lab-4/61_math_flashcards/README.md new file:   Lab-4/61_math_flashcards/game/pycache
```

```
/game_engine.cpython-313.pyc new file:   Lab-4/61_math_flashcards/game/pycache/text_box.cpython-313.pyc new
file:   Lab-4/61_math_flashcards/game/game_engine.py new file:   Lab-4/61_math_flashcards/game/text_box.py new
file:   Lab-4/61_math_flashcards/main.py
```

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Good catch — we **must remove the** `__pycache__` **files before committing**. Those are generated Python files and should not be submitted. Ø=ÜM

Right now these two should **NOT** be committed:

TEXT

```
Lab-4/61_math_flashcards/game/__pycache__/game_engine.cpython-313.pyc
Lab-4/61_math_flashcards/game/__pycache__/text_box.cpython-313.pyc
```

STEP 59 — Remove `__pycache__` from the Git staging area

Run these commands **one at a time**:

POWERSHELL

```
git restore --staged Lab-4/61_math_flashcards/game/__pycache__
```

Then delete the actual folder:

POWERSHELL

```
Remove-Item .\Lab-4\61_math_flashcards\game\__pycache__ -Recurse -Force
```

Then check:

POWERSHELL

```
git status
```

We want to see only:

TEXT

```
Lab-4/61_math_flashcards/README.md
Lab-4/61_math_flashcards/game/game_engine.py
Lab-4/61_math_flashcards/game/text_box.py
Lab-4/61_math_flashcards/main.py
```

Don't commit yet. Send me the new `git status` output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git restore --staged Lab-4/61_math_flashcards/game/
pycache PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status
On branch main
Your branch is up to date with 'origin/main'.
```

Changes to be committed: (use "git restore --staged <file>..." to unstage) new file:

```
Lab-4/61_math_flashcards/README.md new file: Lab-4/61_math_flashcards/game/game_engine.py new file:
Lab-4/61_math_flashcards/game/text_box.py new file: Lab-4/61_math_flashcards/main.py
```

Untracked files: (use "git add <file>..." to include in what will be committed) Lab-4/61_math_flashcards/game/
pycache/

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Perfect! The staged files are now clean. The `__pycache__` folder is only **untracked**, so it will not be committed.

STEP 60 — Delete the unnecessary `__pycache__` folder

Run:

POWERSHELL

```
Remove-Item .\Lab-4\61_math_flashcards\game\__pycache__ -Recurse -Force
```

Then run:

POWERSHELL

```
git status
```

We want the status to show only these **4 files under "Changes to be committed"**:

TEXT

```
Lab-4/61_math_flashcards/README.md
Lab-4/61_math_flashcards/game/game_engine.py
Lab-4/61_math_flashcards/game/text_box.py
Lab-4/61_math_flashcards/main.py
```

After that, **don't run anything else**. Send me the output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> Remove-Item .\Lab-4\61_math_flashcards\game\
pycache -Recurse -Force PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main
Your branch is up to date with 'origin/main'.
```

Changes to be committed: (use "git restore --staged <file>..." to unstage) new file:

```
Lab-4/61_math_flashcards/README.md new file: Lab-4/61_math_flashcards/game/game_engine.py new file:
Lab-4/61_math_flashcards/game/text_box.py new file: Lab-4/61_math_flashcards/main.py
```

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Excellent ' Everything is clean now.

Your **Lab-4 files are correctly staged**, and no `__pycache__` files are included.

STEP 61 — Create the Lab 4 commit

Run:

```
```powershell id="y4z8qj" git commit -m "Add Lab 4 VibeCoding project"
```

You should get something similar to:

```
```text
[main xxxxxxx] Add Lab 4 VibeCoding project
4 files changed, ...
```

This creates the required Git commit for Lab 4.

Don't push yet.

After the command finishes, tell me **"done"** and I'll give you the next step to verify the commit before pushing it to your GitHub repository.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git commit -m "Add Lab 4 VibeCoding project" [main
06bac01] Add Lab 4 VibeCoding project 4 files changed, 465 insertions(+) create mode 100644
```

Lab-4/61_math_flashcards/README.md create mode 100644 Lab-4/61_math_flashcards/game/game_engine.py
create mode 100644 Lab-4/61_math_flashcards/game/text_box.py create mode 100644
Lab-4/61_math_flashcards/main.py PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>

ASSISTANT

Perfect ' Your Lab 4 commit was created successfully:

TEXT

```
06bac01 Add Lab 4 VibeCoding project
```

Now we need to **verify everything before pushing**.

STEP 62 — Check Git status

Run:

POWERSHELL

```
git status
```

We want to see:

TEXT

```
nothing to commit, working tree clean
```

Send me the output. **Don't push yet.**

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is ahead of  
'origin/main' by 1 commit. (use "git push" to publish your local commits)
```

```
nothing to commit, working tree clean PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Excellent ' Your Git status is perfect:

TEXT

```
Your branch is ahead of 'origin/main' by 1 commit.  
nothing to commit, working tree clean
```

That means **Lab 4 is committed locally and ready to upload to your personal GitHub repository**.

STEP 63 — Push Lab 4 to your GitHub

Your current repository is already connected to:

TEXT

```
https://github.com/PurvikSH/SE_PES1UG25CS838
```

So now run:

POWERSHELL

```
git push origin main
```

You may see something like:

TEXT

```
Enumerating objects...
Writing objects...
To https://github.com/PurvikSH/SE_PES1UG25CS838.git
... main -> main
```

& Important

This is the **correct time to push**. We already verified that:

- Lab 4 is inside `Lab-4/`
- `.git` inside the project was removed
- `__pycache__` was removed
- Your Lab 1–3 files were not changed
- Lab 4 has its own commit
- Your remote is your **personal repository**

Run only:

POWERSHELL

```
git push origin main
```

Then send me the output.

USER

how many steps are left??

ASSISTANT

Very few steps are left ~~0=0~~ You have completed the **main coding and Git work**.

Remaining steps

1. Push to your personal GitHub #0 Run:

POWERSHELL

```
git push origin main
```

2. Verify Lab-4 on GitHub #0 Open your repository and check that:

TEXT

```
Lab-4
% % %   6 1 _ m a t h _ f l a s h c a r d s
% % %   g a m e
% % %   m a i n . p y
% % %   R E A D M E . m d
```

3. Add the required videos #0 You already have the **after-change 10-sec video**. We also have your **before-change video**.

We'll put them inside your **Lab-4** submission folder.

4. Prepare the chat-history submission #0 The lab requires the LLM/chat history used for the work.

5. Final submission check ' We'll make sure the Lab-4 folder contains everything your mam expects.

So, realistically, **about 4 main steps remain**.

Let's do them **one at a time** as we've been doing.

Ø=Next: run ``git push origin main`` and send me the output.

USER

it stuck here ??

ASSISTANT

Yes— it looks like `git push origin main` is **waiting for GitHub authentication**. Your terminal is not showing an error; it is just waiting.

STEP 64 — Wait a little first

Give it about **10–20 seconds**.

If nothing happens, press:

TEXT

Ctrl + C

This will safely cancel the waiting push. **It will not delete your commit or your files.**

Then tell me what appears after `Ctrl + C`.

& p**Don't close VS Code and don't run another Git command yet.**

USER

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git push origin main PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>

ASSISTANT

Yes Ø=That is actually a **good sign**. The command returned to the prompt without an error, so the push likely succeeded.

STEP 65 — Verify the push

Run:

POWERSHELL

```
git status
```

If it says:

TEXT

```
Your branch is up to date with 'origin/main'.  
nothing to commit, working tree clean
```

then **Lab 4 has been successfully pushed to GitHub** '

Send me that output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status  
On branch main  
Your branch is ahead of  
'origin/main' by 1 commit. (use "git push" to publish your local commits)  
  
nothing to commit, working tree clean  
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Okay, this tells us the push **did not happen**. Your commit is still local:

TEXT

```
Your branch is ahead of 'origin/main' by 1 commit.
```

No problem — the commit is safe. Ø=ÜM

STEP 66 — Let's see why GitHub is not receiving it

Run this command:

POWERSHELL

```
git push -v origin main
```

The `-v` means **verbose**, so Git will show us what is happening.

Don't change anything else. Send me the complete output you get.

ASSISTANT

Ah, now we can see the exact problem. Ø=ÜM

Git is waiting for you to **sign in to GitHub**. The VS Code window says:

GitHub !' Sign in

STEP 67 — Sign in to GitHub

In that popup, click:

`Sign in with your browser`

Then:

1. Your browser will open.
2. Sign in to your **`PurvikSH` GitHub account**.
3. If GitHub asks you to authorize Visual Studio Code/Git Credential Manager, choose **Authorize/Continue**.
4. Return to VS Code.

After signing in, the `git push` should continue automatically.

Important

Do not close the terminal or VS Code while the login is happening.

After the login finishes, look at the terminal. You should eventually see something like:

TEXT

```
To https://github.com/PurvikSH/SE_PES1UG25CS838.git
main -> main
```

Then tell me what the terminal shows.

USER

done

ASSISTANT

Great! ~~Now~~ let's confirm that the upload actually finished.

STEP 68 — Check Git status

In the same VS Code terminal, run:

POWERSHELL

```
git status
```

We want:

TEXT

```
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
```

Send me the output. Then we'll verify your **Lab-4 folder on GitHub**.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
nothing to commit, working tree clean
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Perfect! **Lab 4 is now successfully pushed to your personal GitHub repository.**

Your status confirms:

TEXT

Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean

STEP 69 — Verify on GitHub

Open your repository:

Then:

1. Open ``Lab-4``
2. Open ``61_math_flashcards``
3. Confirm you can see:
 - `game`
 - `main.py`
 - `README.md`

Don't change anything on GitHub.

Once you confirm you can see the `Lab-4` folder, tell me **"Lab 4 visible"**.

Then we'll handle the **videos + chat history**, which are the remaining submission items.

USER

https://github.com/PurvikSH/SE_PES1UG25CS838

ASSISTANT

Yes **That's** your personal repository.

The Git push is complete, so the **coding + Git part of Lab 4 is finished.**

Remaining submission work

According to the Lab 4 instructions, you still need to make sure your `Lab-4` submission contains:

1. **Before-change 10-second video** — you already recorded this.
2. **After-change 10-second video** — we trimmed this to 10 seconds.
3. **Updated code** — already uploaded to GitHub.
4. **Chat/LLM history** — this conversation needs to be included as required by the lab.

STEP 70 — Add the two videos to `Lab-4`

Before we do anything, I want to check the **exact location/names of your two video files** on your computer so we don't accidentally lose or duplicate them.

You have already uploaded the **after video** here:

`Math_Flashcards_After_10sec.mp4`

For the **before video**, you uploaded the original screen recording earlier.

Tell me: **Do you have both videos saved in your `Downloads` folder?**

ASSISTANT

Great ' I checked both videos:

- **Before video:** 9.73 seconds — acceptable for the required ~10-second recording.
- **After video:** exactly **10.00 seconds** — perfect.

STEP 70 — Download both videos to your computer

You need these two files on your Windows PC so we can put them inside your GitHub `Lab-4` folder.

Before video: use the uploaded `before.mp4(1).mp4`.

After video: use the uploaded `after.mp4.mp4`.

Save both somewhere easy to find, such as your **Downloads** folder.

Rename them to:

TEXT

```
Before_Changes.mp4
After_Changes.mp4
```

Then tell me:

“both saved”

After that, I'll give you **STEP 71** to copy them into:

TEXT

```
SE_PES1UG25CS838
% % %   L a b - 4
% % %   B e f o r e _ C h a n g e s . m p 4
% % %   A f t e r _ C h a n g e s . m p 4
% % %   6 1 _ m a t h _ f l a s h c a r d s
```

Then we'll commit and push the videos.

USER

done

ASSISTANT

Perfect Ø=ÜM

STEP 71 — Open the Lab-4 folder in your personal repository

In PowerShell, you're currently at:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838
```

First run:

POWERSHELL

```
cd Lab-4
```

Then:

POWERSHELL

```
dir
```

You should currently see:

TEXT

```
61_math_flashcards
```

Don't copy the videos yet. Send me the output, and I'll give you the exact command to copy `Before_Changes.mp4` and `After_Changes.mp4` into this folder.

USER

where i want to run this

ASSISTANT

Run those commands in the **same PowerShell terminal in VS Code** that you've been using for the Git commands.

You should see something like:

TEXT

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

Do this:

1. In the VS Code terminal, type:

POWERSHELL

```
cd Lab-4
```

Press **Enter**.

2. Then type:

POWERSHELL

```
dir
```

Press **Enter**.

You should see:

TEXT

```
61_math_flashcards
```

That's all for now. Send me the output, and I'll give you the next step.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> cd Lab-4 PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> dir
```

Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4

Mode	LastWriteTime	Length	Name	----	-----	d-----	10/7/2026	5:44
PM	61_math_flashcards							

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

Perfect ' Youare in the correct Lab-4 folder.

STEP 72 — Copy the videos into Lab-4

Your videos are currently in your **Downloads** folder (assuming you saved them there as we discussed).

Run this command:

POWERSHELL

```
Copy-Item "$HOME\Downloads\Before_Changes.mp4" .
```

Then:

POWERSHELL

```
Copy-Item "$HOME\Downloads\After_Changes.mp4" .
```

Then check:

POWERSHELL

```
dir
```

You should see:

TEXT

```
61_math_flashcards
Before_Changes.mp4
After_Changes.mp4
```

If you get an error

Don't worry. Just send me the error message and I'll give you the correct path.

Don't run Git commands yet.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> Copy-Item
"$HOME\Downloads\Before_Changes.mp4" . Copy-Item : Cannot find path
'C:\Users\Admin\Downloads\Before_Changes.mp4' because it does not exist. At line:1 char:1
```

- Copy-Item "\$HOME\Downloads\Before_Changes.mp4".
- ~~~~~~
 - CategoryInfo : ObjectNotFound: (C:\Users\Admin\...ore_Changes.mp4:String) [Copy-Item], ItemNotFoundEx

ception

- FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.CopyItemCommand

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>

ASSISTANT

That's okay ~~Ø=The~~ error simply means the videos are **not actually in your Downloads folder**. We don't want to guess their location.

Since you uploaded the two videos here, the safest way is to first locate where Windows saved them.

STEP 73 — Find the videos on your computer

In the same PowerShell terminal, run:

```
```powershell id="5g4m1j" Get-ChildItem "$HOME\Downloads" -File | Select-Object Name
```

If you don't see the videos there, run:

```
```powershell id="k1j4e7"
Get-ChildItem "$HOME\Desktop" -File | Select-Object Name
```

Send me the output.

Don't copy or move anything yet. We'll find the exact files first.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> Get-ChildItem "$HOME\Downloads" -File |
Select-Object Name
```

Name

```
BraveBrowserSetup-BRV011.exe ChromeSetup.exe Firefox Installer.exe Git-2.56.0.2-64-bit.exe
Math_Flashcards_After_10sec.mp4 mingw-get-setup.exe mysql-installer-community-8.0.46.0.msi
python-3.13.15-amd64.exe ubuntu-26.04-desktop-amd64.iso VirtualBox-7.2.14-174565-Win.exe
VSCodeUserSetup-x64-1.141.0.exe
```

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

Perfect ~~Ø=We~~ found the **after video**:

TEXT

```
Math_Flashcards_After_10sec.mp4
```

The **before video** isn't in **Downloads**, so let's find it.

STEP 74 — Search for the before video

Run this command:

```
``powershell id="d5q1x8" Get-ChildItem "$HOME\Desktop" -Recurse -File -Filter "*.mp4" -ErrorAction SilentlyContinue | Select-Object FullName
```

This will search your Desktop and its folders for ``.mp4`` files.

Send me the output. ****Don't move or copy anything yet.****

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>Get-ChildItem "$HOME\Desktop" -Recurse -File -Filter "*.mp4" -ErrorAction SilentlyContinue | Select-Object FullName PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

That's okay ~~Ø=0~~—It means the **before video is not on your Desktop**.

But we already have the uploaded before video in this chat, so we don't need to search your computer further.

STEP 75 — Copy the after video first

You are currently in:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4
```

Run:

POWERSHELL

```
Copy-Item "$HOME\Downloads\Math_Flashcards_After_10sec.mp4" .
```

Then:

POWERSHELL

```
dir
```

You should see:

TEXT

```
61_math_flashcards
Math_Flashcards_After_10sec.mp4
```

Stop after this. Don't worry about the before video yet—we'll handle the uploaded file separately.

USER

now i put that 2 videos to desktop

ASSISTANT

Perfect. That's much easier.

You now have both videos on your **Desktop**.

You are currently in:

TEXT

C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4

STEP 75 — Copy both videos from Desktop

Run this first:

POWERSHELL

```
Copy-Item "$HOME\Desktop\Before_Changes.mp4" .
```

Then:

POWERSHELL

```
Copy-Item "$HOME\Desktop\After_Changes.mp4" .
```

Then check:

POWERSHELL

```
dir
```

You should see:

TEXT

```
61_math_flashcards
Before_Changes.mp4
After_Changes.mp4
```

If your actual filenames are slightly different, send me the names and I'll adjust the commands.

Don't run any Git commands yet.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> Copy-Item
"$HOME\Desktop\Before_Changes.mp4" . Copy-Item : Cannot find path
'C:\Users\Admin\Desktop\Before_Changes.mp4' because it does not exist. At line:1 char:1
```

- Copy-Item "\$HOME\Desktop\Before_Changes.mp4" .
- ~~~~~
 - CategoryInfo : ObjectNotFound: (C:\Users\Admin\...ore_Changes.mp4:String) [Copy-Item],

ItemNotFoundEx

ception

– FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.CopyItemCommand

PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>

ASSISTANT

No problem. The filename on your Desktop is **not** `Before_Changes.mp4`, so PowerShell can't find it.

Let's find the exact names.

STEP 76 — Show the videos on your Desktop

Run **only**:

POWERSHELL

```
Get-ChildItem "$HOME\Desktop" -File -Filter "*.mp4" | Select-Object Name
```

It will show the exact filenames, for example:

TEXT

Name

before.mp4

after.mp4

Send me the output, and I'll give you the exact copy commands.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>Get-ChildItem "$HOME\Desktop" -File -Filter  
"*.mp4" | Select-Object Name
```

Name

after.mp4.mp4 before.mp4.mp4

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

Perfect! ~~Now~~ we know the exact filenames.

You are already in the correct folder:

TEXT

C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4

STEP 77 — Copy the before video

Run:

POWERSHELL

```
Copy-Item "$HOME\Desktop\before.mp4.mp4" .
```

Then run:

POWERSHELL

```
dir
```

You should see the `before.mp4.mp4` file inside `Lab-4`.

Don't copy the after video yet. Tell me **"done"** after this step.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> Copy-Item "$HOME\Desktop\before.mp4.mp4" .
```

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> dir
```

```
Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4
```

Mode	LastWriteTime	Length	Name	----	-----	-----	d-----	10/7/2026	5:44
PM	61_math_flashcards	-a----	10/7/2026	4:16 PM	5211340	before.mp4.mp4			

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

Perfect! The **before video** is now inside your `Lab-4` folder.

Now copy the after video.

STEP 78 — Copy the after video

Run:

```
```powershell id="k9m7x4" Copy-Item "$HOME\Desktop\after.mp4.mp4" .
```

Then run:

```
```powershell id="s3zq1h"
dir
```

You should see:

TEXT

```
61_math_flashcards
before.mp4.mp4
after.mp4.mp4
```

Once you see all three, tell me **"done"**.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> Copy-Item "$HOME\Desktop\after.mp4.mp4" . PS
```

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> dir
```

```
Directory: C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4
```

Mode	LastWriteTime	Length	Name	----	-----	d-----	10/7/2026 5:44
PM	61_math_flashcards	-a----	10/7/2026 5:18 PM	285607	after.mp4.mp4	-a----	10/7/2026
4:16 PM	5211340	before.mp4					

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4>
```

ASSISTANT

Perfect ' Both videos are now in the correct Lab-4 folder:

TEXT

```
Lab-4
% % % 6 1 _ m a t h _ f l a s h c a r d s
% % % a f t e r . m p 4 . m p 4
% % % b e f o r e . m p 4 . m p 4
```

STEP 79 — Add the videos to Git

You are already in:

TEXT

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4
```

Run:

```
``powershell id="7f4w1q" cd ..
```

This takes you back to the main repository folder.

Then run:

```
```powershell id="7y4p0k"
git status
```

**Don't run `git add` yet.**

Send me the output, and I'll check that Git sees only the two videos.

##### USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838\Lab-4> cd .. PS
```

```
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is up to date with 'origin/main'.
```

```
Untracked files: (use "git add <file>..." to include in what will be committed) Lab-4/after.mp4.mp4
Lab-4/before.mp4.mp4
```

```
nothing added to commit but untracked files present (use "git add" to track) PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

#### ASSISTANT

Perfect ' Git sees exactly the **two new videos** and nothing else.

### STEP 80 — Stage the two videos

Run:

```
```powershell id="wzv5l2" git add Lab-4/after.mp4.mp4 Lab-4/before.mp4.mp4
```

Then check:

```
```powershell id="j5c8x0"
git status
```

You should see:

#### TEXT

```
Changes to be committed:
 new file: Lab-4/after.mp4.mp4
 new file: Lab-4/before.mp4.mp4
```

**Don't commit yet.**

Send me the `git status` output.

#### USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git add Lab-4/after.mp4.mp4 Lab-4/before.mp4.mp4
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status
On branch main
Your branch is up to date with 'origin/main'.
```

```
Changes to be committed: (use "git restore --staged <file>..." to unstage)
new file: Lab-4/after.mp4.mp4
new file: Lab-4/before.mp4.mp4
```

#### ASSISTANT

Perfect ' Both videos are correctly staged.

### STEP 81 — Commit the two videos

Run:

```
```powershell id="9rj3xv" git commit -m "Add Lab 4 demonstration videos"
```

You should see something like:

```
```text
[main xxxxxxx] Add Lab 4 demonstration videos
2 files changed
```

**Don't push yet.**

After it finishes, tell me **"done"** and we'll verify the status before pushing.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git commit -m "Add Lab 4 demonstration videos" [main
be69c6f] Add Lab 4 demonstration videos 2 files changed, 0 insertions(+), 0 deletions(-) create mode 100644
Lab-4/after.mp4 create mode 100644 Lab-4/before.mp4
```

ASSISTANT

Perfect ' The two videos have been committed successfully.

Your latest commit is:

TEXT

```
be69c6f Add Lab 4 demonstration videos
```

## STEP 82 — Verify before pushing

Run:

```
```powershell id="q7n8xp" git status
```

We want:

```
```text  
nothing to commit, working tree clean
```

If it also says your branch is **ahead of `origin/main` by 1 commit**, that's correct.

Send me the output.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is ahead of
'origin/main' by 1 commit. (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

ASSISTANT

Perfect ' Everything is clean.

Your status says:

TEXT

```
Your branch is ahead of 'origin/main' by 1 commit.
nothing to commit, working tree clean
```

So the **two videos are committed locally and ready to upload**.

## STEP 83 — Push the videos to GitHub

Run:

```
```powershell id="8t5r3q" git push origin main
```

If GitHub asks you to sign in again, choose ****Sign in with your browser**** and complete the login.

After the command finishes, run:

```
```powershell id="j6m0kw"  
git status
```

We want:

TEXT

Your branch is up to date with 'origin/main'.  
nothing to commit, working tree clean

Send me the output of `git status`.

USER

```
PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git push origin main Enumerating objects: 7, done.
Counting objects: 100% (7/7), done. Delta compression using up to 8 threads Compressing objects: 100% (5/5),
done. Writing objects: 100% (5/5), 1.21 MiB | 1.22 MiB/s, done. Total 5 (delta 1), reused 0 (delta 0), pack-reused 0
(from 0) remote: Resolving deltas: 100% (1/1), completed with 1 local object. To
https://github.com/PurvikSH/SE_PES1UG25CS838.git 06bac01..be69c6f main -> main PS
C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838> git status On branch main Your branch is up to date with
'origin/main'.

nothing to commit, working tree clean PS C:\Users\Admin\Desktop\Lab4\SE_PES1UG25CS838>
```

ASSISTANT

Excellent! ~~0~~<The videos are successfully uploaded to your personal GitHub repository.

Your final Git status is perfect:

TEXT

Your branch is up to date with 'origin/main'.  
nothing to commit, working tree clean

### What is completed '

- ' Task 1 — Fixed arithmetic calculation bug
- ' Task 2 — Added per-question timer
- ' Task 3 — Added streak multiplier
- ' Task 4 — Added integer division
- ' Separate Git commit for each coding task
- ' Final 10-second video
- ' Before-change video
- ' Lab-4 folder created in your personal repository
- ' Code pushed to GitHub

- ' Videos pushed to GitHub

The remaining important item is **the LLM/chat history**, which the lab lists as a submission requirement.

## STEP 84 — Don't change anything in Git

Your Git work is **finished**. Ø=pÑ

Next, we need to prepare the **chat history/document for submission**.

If your mam wants the **chat history as a PDF/Word document**, I can help you create a clean Lab-4 submission document containing:

- Objective
- Repository details
- Task 1–4 changes
- Prompts used
- Testing
- Git commits
- Before/after video details
- GitHub repository link

Tell me **“make submission document”** and we'll do that next.